

Lecture 2: Traits

Methods, traits, functional programming and standard library

Willem Vanhulle

DevLab Rust 2025

Tuesday November 11, 2025

github.com/wvhulle/rust-course-ghent

1. Quiz about last lecture	1
1.1. Comparing pointers	2
1.2. Patterns	4
1.3. Bool	6
2. Pattern matching	8
3. Tooling intermezzo	25
4. Methods and traits	29
5. Generics	48
6. Closures	69
7. Standard library types	85
8. Conclusion	111



1.1. Comparing pointers

1.1.1. Question

```
1 struct S;  
2  
3 fn main() {  
4     let [x, y] = &mut [S, S];  
5  
6     let eq = x as *mut S == y as *mut S;  
7     print!("{}", eq as u8);  
8 }
```

(playground link)

What does this program print?

1.1. Comparing pointers

1.1.1. Question

```
1 struct S;  
2  
3 fn main() {  
4     let [x, y] = &mut [S, S];  
5  
6     let eq = x as *mut S == y as *mut S;  
7     print!("{}", eq as u8);  
8 }
```

(playground link)

What does this program print?

...

1.1. Comparing pointers

1.1.2. Explanation

What are zero-sized types?

1. Quiz about last lecture

1.1. Comparing pointers

1.1.2. Explanation

What are zero-sized types?

Types that occupy no space at runtime.

```
1 struct S;  
2 fn main() {  
3     let [x, y] = &mut [S, S];  
4     let eq = x as *mut S == y as *mut S;  
5     print!("{}", eq as u8);  
6 }
```

[\(playground link\)](#)

1.1. Comparing pointers

1.1.2. Explanation

What are zero-sized types?

Types that occupy no space at runtime.

```
1 struct S;  
2 fn main() {  
3     let [x, y] = &mut [S, S];  
4     let eq = x as *mut S == y as *mut S;  
5     print!("{}", eq as u8);  
6 }
```

[\(playground link\)](#)

Desugaring array
pattern match

```
1 fn main() {  
2     let tmp: [S; 2] = [S, S];  
3     let x = &mut tmp[0];  
4     let y = &mut tmp[1];  
5     let eq = x as *mut S == y as *mut S;  
6     print!("{}", eq as u8);  
7 }
```

[\(playground link\)](#)

1.1. Comparing pointers

1.1.2. Explanation

What are zero-sized types?

Types that occupy no space at runtime.

```
1 struct S;  
2 fn main() {  
3     let [x, y] = &mut [S, S];  
4     let eq = x as *mut S == y as *mut S;  
5     print!("{}", eq as u8);  
6 }
```

(playground link)

Desugaring array
pattern match

Operator == precedence
above cast with as

```
1 fn main() {  
2     let tmp: [S; 2] = [S, S];  
3     let x = &mut tmp[0];  
4     let y = &mut tmp[1];  
5     let eq = x as *mut S == y as *mut S;  
6     print!("{}", eq as u8);  
7 }
```

(playground link)

```
1 fn main() {  
2     let tmp: [S; 2] = [S, S];  
3     let x = &mut tmp[0];  
4     let y = &mut tmp[1];  
5     let eq = (x as *mut S) == (y as *mut S);  
6     print!("{}", eq as u8);  
7 }
```

(playground link)

1.1. Comparing pointers

1.1.2. Explanation

What are zero-sized types?

Types that occupy no space at runtime.

```
1 struct S;
2 fn main() {
3     let [x, y] = &mut [S, S];
4     let eq = x as *mut S == y as *mut S;
5     print!("{}", eq as u8);
6 }
```

[\(playground link\)](#)

Desugaring array
pattern match

```
1 fn main() {
2     let tmp: [S; 2] = [S, S];
3     let x = &mut tmp[0];
4     let y = &mut tmp[1];
5     let eq = x as *mut S == y as *mut S;
6     print!("{}", eq as u8);
7 }
```

[\(playground link\)](#)

Operator == precedence
above cast with as

```
1 fn main() {
2     let tmp: [S; 2] = [S, S];
3     let x = &mut tmp[0];
4     let y = &mut tmp[1];
5     let eq = (x as *mut S) == (y as *mut S);
6     print!("{}", eq as u8);
7 }
```

[\(playground link\)](#)

Explicit casts

```
1 fn main() {
2     let tmp: [S; 2] = [S, S];
3     let x = &mut tmp[0];
4     let y = &mut tmp[1];
5     let x_ptr: *mut S = x;
6     let y_ptr: *mut S = y;
7     let eq = x_ptr == y_ptr;
8     print!("{}", eq as u8);
9 }
```

[\(playground link\)](#)

Are x_ptr and y_ptr equal?

1.1. Comparing pointers

1.1.2. Explanation

What are zero-sized types?

Types that occupy no space at runtime.

```
1 struct S;
2 fn main() {
3     let [x, y] = &mut [S, S];
4     let eq = x as *mut S == y as *mut S;
5     print!("{}", eq as u8);
6 }
```

(playground link)

Desugaring array
pattern match

```
1 fn main() {
2     let tmp: [S; 2] = [S, S];
3     let x = &mut tmp[0];
4     let y = &mut tmp[1];
5     let eq = x as *mut S == y as *mut S;
6     print!("{}", eq as u8);
7 }
```

(playground link)

Operator == precedence
above cast with as

```
1 fn main() {
2     let tmp: [S; 2] = [S, S];
3     let x = &mut tmp[0];
4     let y = &mut tmp[1];
5     let eq = (x as *mut S) == (y as *mut S);
6     print!("{}", eq as u8);
7 }
```

(playground link)

Explicit casts

```
1 fn main() {
2     let tmp: [S; 2] = [S, S];
3     let x = &mut tmp[0];
4     let y = &mut tmp[1];
5     let x_ptr: *mut S = x;
6     let y_ptr: *mut S = y;
7     let eq = x_ptr == y_ptr;
8     print!("{}", eq as u8);
9 }
```

(playground link)

Are x_ptr and y_ptr equal?

Yes, since the compiler can see that S is a zero-sized type.

1.2. Patterns

1.2.1. Question

Last session I mentioned that variable assignments are actually patterns.

```
1 fn main() {  
2     let (_, x, y) = (0, 1, ..);  
3     print!("{}", b"066"[y][x]);  
4 }
```

(playground link)

What does this program print?

1.2. Patterns

1.2.1. Question

Last session I mentioned that variable assignments are actually patterns.

```
1 fn main() {  
2     let (_, x, y) = (0, 1, ..);  
3     print!("{}", b"066"[y][x]);  
4 }
```

(playground link)

What does this program print?

...

1.2.2. Explanation

```
1 fn main() {  
2     let (_, x, y) = (0,  
3     1, ..);  
4     print!("{}", b"066"[y][x]);  
5 }
```

(playground link)

1.2. Patterns

1.2.2. Explanation

```
1 fn main() {  
2     let (.., x, y) = (0,  
3         1, ..);  
4     print!("{}", b"066"[y][x]);  
5 } (playground link)
```

Desugaring ..
as a range

```
1 fn main() {  
2     let x = 1;  
3     let y: RangeFull = ..;  
4     print!("{}", b"066"[y][x]);  
5 } (playground link)
```

1.2. Patterns

1.2.2. Explanation

```
1 fn main() {  
2     let (_, x, y) = (0,  
3         1, ..);  
4     print!("{}", b"066"[y][x]);  
                                     (playground link)  
}
```

Desugaring ..
as a range

```
1 fn main() {  
2     let x = 1;  
3     let y: RangeFull = ..;  
4     print!("{}", b"066"[y][x]);  
5 }  
                                     (playground link)
```

Explicit type
binary literal

```
1 fn main() {  
2     let x = 1;  
3     let y = ..;  
4     let bytes: &'static [u8; 3] =  
5         b"066";  
6     print!("{}", bytes[..][x]);  
                                     (playground link)  
}
```

1.2. Patterns

1.2.2. Explanation

```
1 fn main() {
2     let (_, x, y) = (0,
3         1, ..);
4     print!("{}", b"066"[y][x]);
5 }
```

(playground link)

Desugaring ..
as a range

```
1 fn main() {
2     let x = 1;
3     let y: RangeFull = ..;
4     print!("{}", b"066"[y][x]);
5 }
```

(playground link)

Explicit type
binary literal

```
1 fn main() {
2     let x = 1;
3     let y = ..;
4     let bytes: &'static [u8; 3] =
5         b"066";
6     print!("{}", bytes[..][x]);
7 }
```

(playground link)

Slice
indexing

```
1 fn main() {
2     let x = 1;
3     let bytes = b"066";
4     print!("{}", bytes[x]);
5 }
```

(playground link)

What is the decimal representation of the ASCII character '6'?

1.2. Patterns

1.2.2. Explanation

```
1 fn main() {
2     let (_, x, y) = (0,
3         1, ..);
4     print!("{}", b"066"[y][x]);
4 }
```

(playground link)

Desugaring ..
as a range

```
1 fn main() {
2     let x = 1;
3     let y: RangeFull = ..;
4     print!("{}", b"066"[y][x]);
5 }
```

(playground link)

Explicit type
binary literal

```
1 fn main() {
2     let x = 1;
3     let y = ..;
4     let bytes: &'static [u8; 3] =
5         b"066";
6     print!("{}", bytes[..][x]);
6 }
```

(playground link)

Slice
indexing

```
1 fn main() {
2     let x = 1;
3     let bytes = b"066";
4     print!("{}", bytes[x]);
5 }
```

(playground link)

What is the decimal representation of the ASCII character '6'?

54

1.3. Bool

1.3.1. Question

```
1  fn check(x: i32) -> bool {
2      print!("{}", x);
3      false
4  }
5
6  fn main() {
7      match (1, 2) {
8          (x, _) | (_, x) if check(x) => {
9              print!("3")
10             }
11          _ => print!("4"),
12      }
13 }
```

[\(playground link\)](#)

What does this program print?

1.3. Bool

1.3.1. Question

```
1  fn check(x: i32) -> bool {  
2      print!("{}", x);  
3      false  
4  }  
5  
6  fn main() {  
7      match (1, 2) {  
8          (x, _) | (_, x) if check(x) => {  
9              print!("3")  
10             }  
11             _ => print!("4"),  
12         }  
13 }
```

(playground link)

What does this program print?

...

1.3. Bool

1.3.2. Explanation

1. Quiz about last lecture

```
1 fn check(x: i32) -> bool {  
2     print!("{}", x);  
3     false  
4 } (playground link)
```

1.3. Bool

1.3.2. Explanation

1. Quiz about last lecture

```
1 fn check(x: i32) -> bool {  
2     print!("{}", x);  
3     false  
4 } (playground link)
```



```
1 fn main() {  
2     match (1, 2) {  
3         (x, _) | (_, x) if check(x) =>  
4             {  
5                 print!("3")  
6             }  
7         _ => print!("4"),  
8     } (playground link)
```

1.3. Bool

1.3.2. Explanation

```
1 fn check(x: i32) -> bool {  
2     print!("{}", x);  
3     false  
4 } (playground link)
```



```
1 fn main() {  
2     match (1, 2) {  
3         (x, _) | (_, x) if check(x) =>  
4             {  
5                 print!("3")  
6             }  
7         _ => print!("4"),  
8     } (playground link)
```



```
1 fn main() {  
2     match (1, 2) {  
3         (x, _) | (_, x) if check(x)  
4             => {  
5                 print!("3")  
6             }  
7         _ => print!("4"),  
8     } (playground link)
```

1.3. Bool

1.3.2. Explanation

```
1 fn check(x: i32) -> bool {  
2     print!("{}", x);  
3     false  
4 } (playground link)
```

```
1 fn main() {  
2     match (1, 2) {  
3         (x, _) | (_, x) if check(x) =>  
4             {  
5                 print!("3")  
6             }  
7         _ => print!("4"),  
8     } (playground link)
```

```
1 fn main() {  
2     match (1, 2) {  
3         (x, _) | (_, x) if check(x)  
4             => {  
5                 print!("3")  
6             }  
7         _ => print!("4"),  
8     } (playground link)
```

```
1 fn check(x: i32) -> bool {  
2     print!("{}", x);  
3     false  
4 } (playground link)
```

1.3. Bool

1.3.2. Explanation

```
1 fn check(x: i32) -> bool {
2     print!("{}", x);
3     false
4 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x) =>
4             print!("3")
5     }
6     _ => print!("4"),
7 }
8 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x)
4             => {
5                 print!("3")
6             }
7         _ => print!("4"),
8     }
```

(playground link)

```
1 fn check(x: i32) -> bool {
2     print!("{}", x);
3     false
4 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x)
4             => {
5                 print!("3")
6             }
7         _ => print!("4"),
8     }
```

(playground link)

1.3. Bool

1.3.2. Explanation

```
1 fn check(x: i32) -> bool {
2     print!("{}", x);
3     false
4 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x) =>
4             print!("3")
5     }
6     _ => print!("4"),
7 }
8 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x)
4             => {
5                 print!("3")
6             }
7     }
8 }
```

(playground link)

```
1 fn check(x: i32) -> bool {
2     print!("{}", x);
3     false
4 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x)
4             => {
5                 print!("3")
6             }
7     }
8 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x) =>
4             {
5                 print!("3")
6             }
7     }
8 }
```

(playground link)

What is going on here?

1.3. Bool

1.3.2. Explanation

```
1 fn check(x: i32) -> bool {
2     print!("{}", x);
3     false
4 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x) =>
4             print!("3")
5     }
6     _ => print!("4"),
7 }
8 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x)
4             => {
5                 print!("3")
6             }
7     }
8 }
```

(playground link)

```
1 fn check(x: i32) -> bool {
2     print!("{}", x);
3     false
4 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x)
4             => {
5                 print!("3")
6             }
7     }
8 }
```

(playground link)

```
1 fn main() {
2     match (1, 2) {
3         (x, _) | (_, x) if check(x) =>
4             {
5                 print!("3")
6             }
7     }
8 }
```

(playground link)

What is going on here?

the guard is being run multiple times, once per |-separated alternative in the match-arm.

1. Quiz about last lecture	1
2. Pattern matching	8
2.1. Irrefutable patterns	9
2.2. Matching values	11
2.3. Practice: Rustlings Enums	15
2.4. Structs	16
2.5. Enums	21
2.6. Let control flow	22
2.7. Practice: Exercism Clock	23
2.8. Exercise: Expression evaluation	24
3. Tooling intermezzo	25
4. Methods and traits	29
5. Generics	48
6. Closures	69
7. Standard library types	85
8. Conclusion	111

2.1. Irrefutable patterns

Info

An **irrefutable pattern** is a pattern that always matches.

```
1  fn takes_tuple(tuple: (char, i32, bool)) {  
2      let a = tuple.0;  
3      let b = tuple.1;  
4      let c = tuple.2;  
5  
6      // This does the same thing as above.  
7      let (a, b, c) = tuple;  
8  
9      // Ignore the first element, only bind the  
10     second and third.  
11  
12     // Ignore everything but the last element.  
13     let (_, c) = tuple;  
14 }
```

(playground link)

2.1. Irrefutable patterns

Info

An **irrefutable pattern** is a pattern that always matches.

```
1  fn takes_tuple(tuple: (char, i32, bool)) {  
2      let a = tuple.0;  
3      let b = tuple.1;  
4      let c = tuple.2;  
5  
6      // This does the same thing as above.  
7      let (a, b, c) = tuple;  
8  
9      // Ignore the first element, only bind the  
10     second and third.  
11  
12     // Ignore everything but the last element.  
13     let (_, c) = tuple;  
14 }
```

(playground link)

What happens when adding or removing an element to the tuple and look at the resulting compiler errors

2.1. Irrefutable patterns

Info

An **irrefutable pattern** is a pattern that always matches.

```
1 fn takes_tuple(tuple: (char, i32, bool)) {  
2     let a = tuple.0;  
3     let b = tuple.1;  
4     let c = tuple.2;  
5  
6     // This does the same thing as above.  
7     let (a, b, c) = tuple;  
8  
9     // Ignore the first element, only bind the  
10    second and third.  
11  
12    // Ignore everything but the last element.  
13    let (_, c) = tuple;  
14 }
```

(playground link)

What happens when adding or removing an element to the tuple and look at the resulting compiler errors

The destructuring pattern must match the structure of the value exactly.

2.1. Irrefutable patterns

Info

An **irrefutable pattern** is a pattern that always matches.

```
1  fn takes_tuple(tuple: (char, i32, bool)) {  
2      let a = tuple.0;  
3      let b = tuple.1;  
4      let c = tuple.2;  
5  
6      // This does the same thing as above.  
7      let (a, b, c) = tuple;  
8  
9      // Ignore the first element, only bind the  
10     second and third.  
11  
12     // Ignore everything but the last element.  
13     let (_, c) = tuple;  
14 }
```

(playground link)

What happens when adding or removing an element to the tuple and look at the resulting compiler errors

The destructuring pattern must match the structure of the value exactly.

Info

A variable binding is actually a pattern itself!

The `_` pattern matches anything.

2.1. Irrefutable patterns

2.1.1. Advanced usage of ..

Ignoring middle element:

```
1 fn takes_tuple(tuple: (char, i32, bool, u8)) {  
2     let (first, .., last) = tuple;  
3 }
```

([playground link](#))

2.1. Irrefutable patterns

2.1.1. Advanced usage of ..

Ignoring middle element:

```
1 fn takes_tuple(tuple: (char, i32, bool, u8)) {  
2     let (first, .., last) = tuple;  
3 }
```

([playground link](#))

Works with arrays as well:

```
1 fn takes_array(array: [u8; 5]) {  
2     let [first, .., last] = array;  
3 }
```

([playground link](#))

2.1. Irrefutable patterns

2.1.1. Advanced usage of ..

Ignoring middle element:

```
1 fn takes_tuple(tuple: (char, i32, bool, u8)) {  
2     let (first, .., last) = tuple;  
3 }
```

(playground link)

Works with arrays as well:

```
1 fn takes_array(array: [u8; 5]) {  
2     let [first, .., last] = array;  
3 }
```

(playground link)

Warning

Pattern matching works on all data types in Rust!

2.2. Matching values

Patterns can also be simple values like characters:

```
1  #[rustfmt::skip]
2  fn main() {
3      let input = 'x';
4      match input {
5          'q'                => println!("Quitting"),
6          'a' | 's' | 'w' | 'd' => println!("Moving around"),
7          '0'..'9'           => println!("Number input"),
8          key if key.is_lowercase() => println!("Lowercase: {key}"),
9          _                  => println!("Something else"),
10     }
11 }
```

(playground link)

A variable in the pattern (key in this example) will create a binding that can be used within the match arm.

2.2. Matching values

A **match guard**:

- causes the arm to match only if the condition is true.
- If the condition is false the match will continue checking later cases.

```
1  #[rustfmt::skip]
2  fn main() {
3      let input = 'a';
4      match input {
5          key if key.is_uppercase() => println!("Uppercase"),
6          key => if input == 'q' { println!("Quitting") },
7          _   => println!("Lowercase"),
8      }
9  }
```

(playground link)

What is the bug in this program?

2.2. Matching values

A **match guard**:

- causes the arm to match only if the condition is true.
- If the condition is false the match will continue checking later cases.

```
1 #[rustfmt::skip]
2 fn main() {
3     let input = 'a';
4     match input {
5         key if key.is_uppercase() => println!("Uppercase"),
6         key => if input == 'q' { println!("Quitting") },
7         _   => println!("Lowercase"),
8     }
9 }
```

(playground link)

What is the bug in this program?

The second arm will always match, so the third arm is unreachable. Write match guard in front of the second arm: `key if key == 'q' => ...`

Warning

Can't use an existing variable as the condition in a match arm, as it will instead be interpreted as a variable name pattern, which creates a new variable that will shadow the existing one.

```
1 let expected = 5;
2 match 123 {
3     expected => println!("Expected value is 5, actual is {expected}"),
4     _ => println!("Value was something else"),
5 }
```

[\(playground link\)](#)

What is the output of this program?

Warning

Can't use an existing variable as the condition in a match arm, as it will instead be interpreted as a variable name pattern, which creates a new variable that will shadow the existing one.

```
1 let expected = 5;
2 match 123 {
3     expected => println!("Expected value is 5, actual is {expected}"),
4     _ => println!("Value was something else"),
5 }
```

[\(playground link\)](#)

What is the output of this program?

Will always print “Expected value is 5, actual is 123”.

2.2. Matching values

Use @ to bind to a value while also testing it:

```
1 let expected = 5;
2 match 123 {
3     val @ 1..=10 => println!("Value {val} is between 1 and 10"),
4     val @ 11..=20 => println!("Value {val} is between 11 and 20"),
5     val => println!("Value {val} is something else"),
6 }
```

(playground link)

2.3. Practice: Rustlings Enums

Practice basic enums and pattern matching:

Rustlings exercises:

- `exercises/08_enums/enums1.rs` - Define basic enums
- `exercises/08_enums/enums2.rs` - Enums with associated data
- `exercises/08_enums/enums3.rs` - Pattern matching with enums

Online: <https://github.com/rust-lang/rustlings>

Like tuples, structs can also be deconstructed by matching:

```
1  struct Foo {
2      x: (u32, u32),
3      y: u32,
4  }
5
6  #[rustfmt::skip]
7  fn main() {
8      let foo = Foo { x: (1, 2), y: 3 };
9      match foo {
10         Foo { y: 2, x: i }    => println!("y = 2, x = {i:?}"),
11         Foo { x: (1, b), y } => println!("x.0 = 1, b = {b}, y = {y}"),
12         Foo { y, .. }        => println!("y = {y}, other fields were ignored"),
13     }
14 }
```

(playground link)

Add or remove fields to Foo and see what happens!

2.4. Structs

2.4.1. Reference patterns

When matching on a reference, Rust **automatically dereferences** it for you:

```
1 fn main() {  
2     let foo = Foo { x: (1, 2), y: 3 };  
3     match &foo { // `foo` is turned into a reference  
4         Foo { y: 2, x: i } => println!("y = 2, x = {i:?}"),  
5         Foo { x: (1, b), y } => println!("x.0 = 1, b = {b}, y = {y}"),  
6         Foo { y, .. }      => println!("y = {y}, other fields were ignored"),  
7     }  
8 }
```

(playground link)

Info

The pattern `Foo { ... }` works on `&foo` because Rust automatically dereferences. You could also write `&Foo { ... }` explicitly, but it's not required.

Two equivalent ways to match on a reference:

```
1 match &foo {  
2     Foo { y, .. } => println!("y = {y}"), // Implicit dereference  
3 }  
4  
5 match &foo {  
6     &Foo { y, .. } => println!("y = {y}"), // Explicit dereference pattern  
7 } (playground link)
```

Pick the one you like best!

Warning

In the implicit version, bound variables like `y` will be references. In the explicit version with `&Foo`, `y` will be a value (copied from the struct).

2.4. Structs

2.4.2. Mutable reference patterns

When matching on `&mut`, bound variables are mutable references:

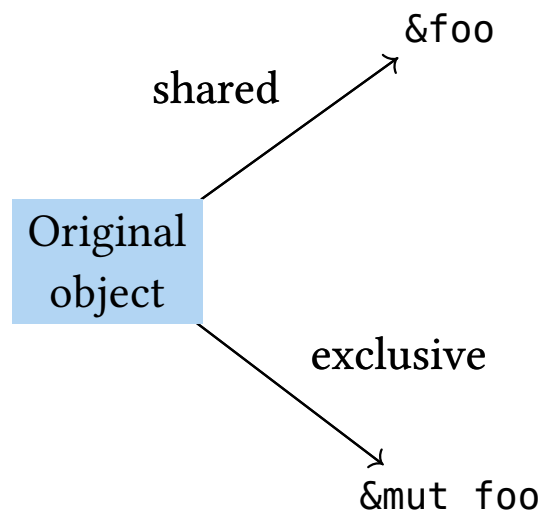
```
1  struct Foo {  
2      x: (u32, u32),  
3      y: u32,  
4  }  
5  fn main() {  
6      let mut foo = Foo { x: (1, 2), y: 3 };  
7      match &mut foo {  
8          Foo { x: (1, 2), y } => *y = 4, // y is &mut u32, need * to assign  
9          Foo { y, .. }      => println!("y = {y}"),  
10     }  
11 }
```

(playground link)

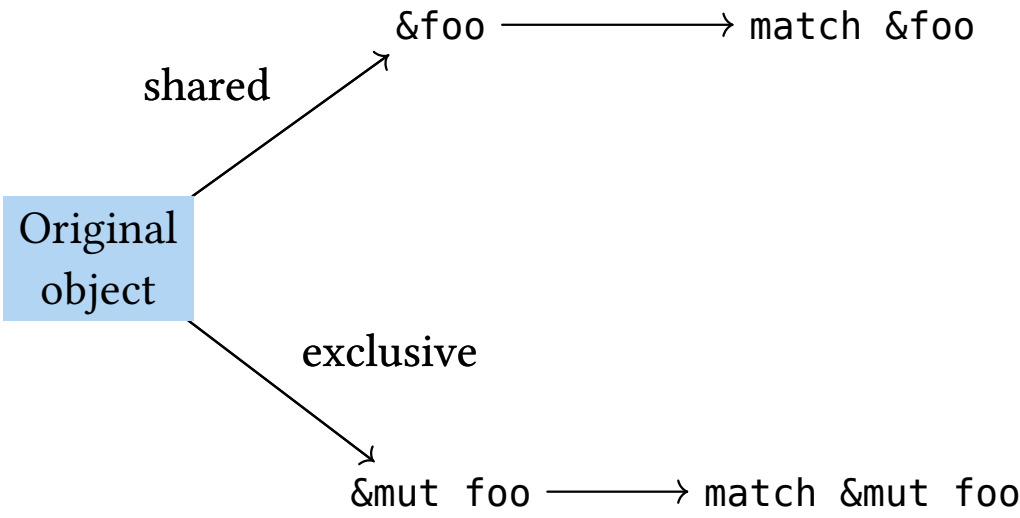
Warning

Bound variables like `y` are `&mut u32`, not `u32`. You must dereference with `*` to assign values.

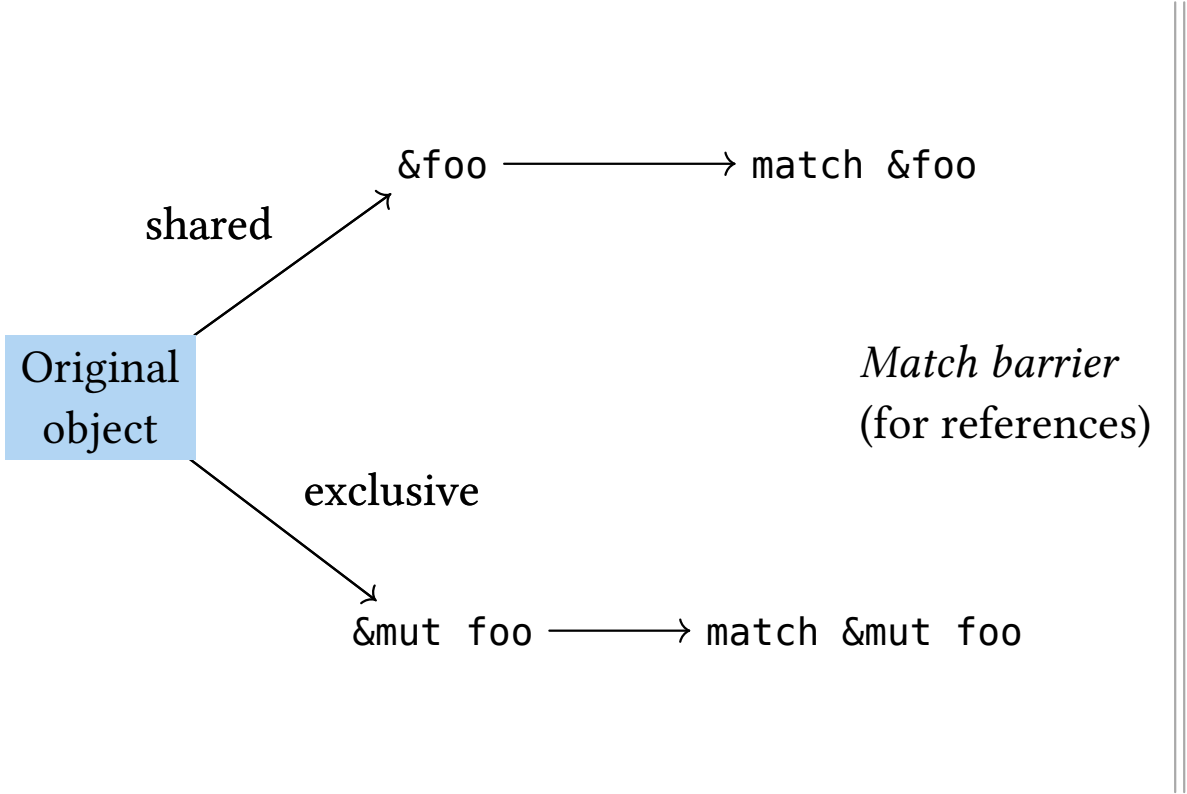
2.4.3. Intuition



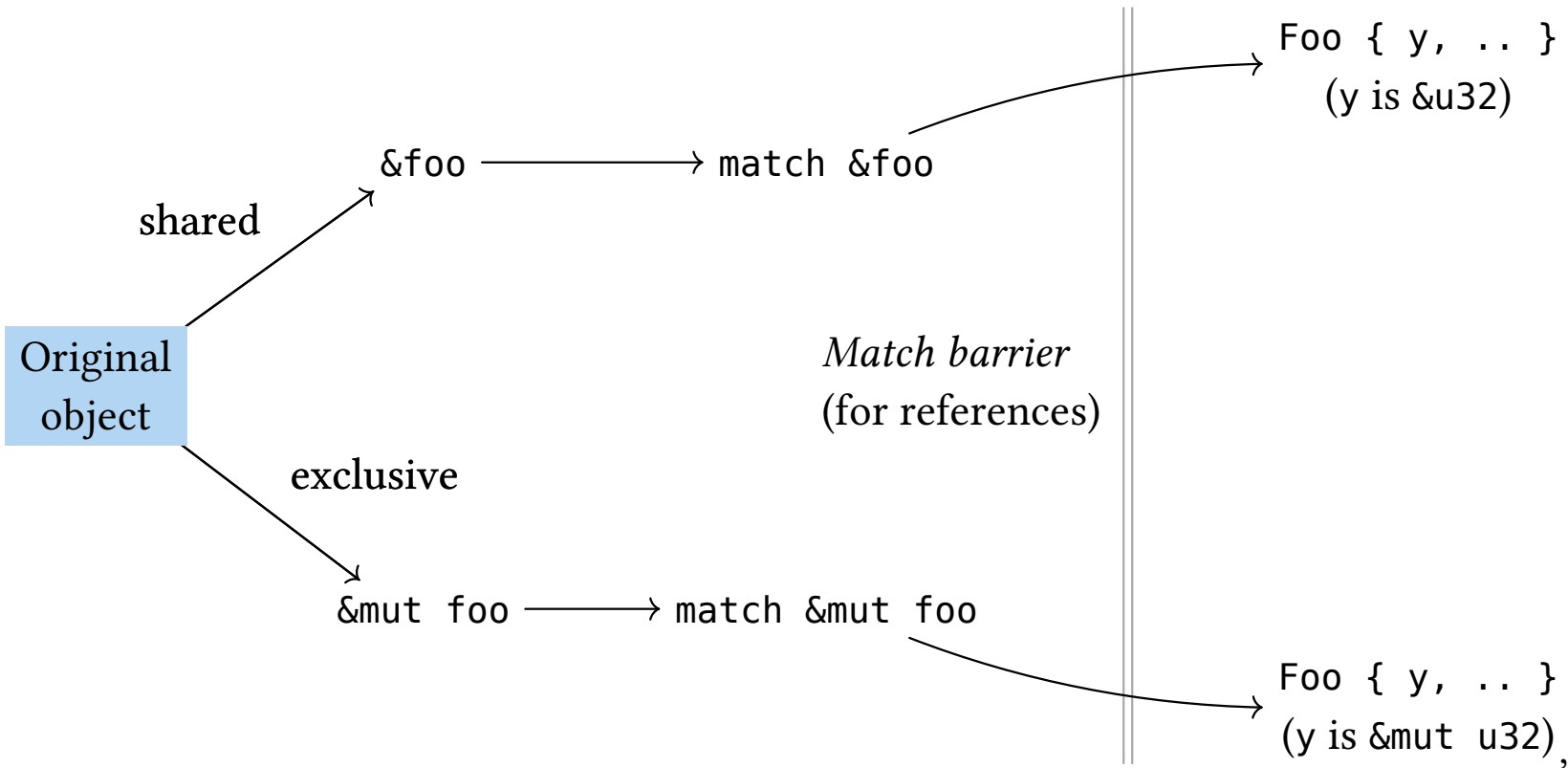
2.4.3. Intuition



2.4.3. Intuition



2.4.3. Intuition



2.5. Enums

Like tuples, enums can also be deconstructed by matching:

```
1  enum Result {  
2      Ok(i32),  
3      Err(String),  
4  }  
5  
6  fn divide_in_two(n: i32) -> Result {  
7      if n % 2 == 0 {  
8          Result::Ok(n / 2)  
9      } else {  
10         Result::Err(format!("cannot divide {n} into two equal parts"))  
11     }  
12 }
```

[\(playground link\)](#)

2.5. Enums

Like tuples, enums can also be deconstructed by matching:

```
1  enum Result {  
2      Ok(i32),  
3      Err(String),  
4  }  
5  
6  fn divide_in_two(n: i32) -> Result {  
7      if n % 2 == 0 {  
8          Result::Ok(n / 2)  
9      } else {  
10         Result::Err(format!("cannot divide {n} into two equal parts"))  
11     }  
12 }
```

[\(playground link\)](#)

```
1  fn main() {  
2      let n = 100;  
3      match divide_in_two(n) {  
4          Result::Ok(half) => println!("{n} divided in two is {half}"),  
5          Result::Err(msg) => println!("sorry, an error happened: {msg}"),  
6      }  
7  }
```

[\(playground link\)](#)

Warning

Rust does not allow non-exhaustive matches

2.6. Let control flow

```
1  use std::time::Duration;
2
3  fn sleep_for(secs: f32) {
4      let result = Duration::try_from_secs_f32(secs);
5
6      if let Ok(duration) = result {
7          std::thread::sleep(duration);
8          println!("slept for {duration:?}");
9      }
10 }
11
12 fn main() {
13     sleep_for(-10.0);
14     sleep_for(0.8);
15 }
```

(playground link)

When to use if let over match?

2.6. Let control flow

```
1 use std::time::Duration;
2
3 fn sleep_for(secs: f32) {
4     let result = Duration::try_from_secs_f32(secs);
5
6     if let Ok(duration) = result {
7         std::thread::sleep(duration);
8         println!("slept for {duration:?}");
9     }
10 }
11
12 fn main() {
13     sleep_for(-10.0);
14     sleep_for(0.8);
15 }
```

(playground link)

When to use if let over match?

Use if let when you only care about one specific pattern and want to ignore all others.

Exercism: Clock:

- Path: `exercises/practice/clock/`
- Goal: Implement a clock that handles times without dates
- Requires: Structs, methods, pattern matching with `match`

Online: <https://exercism.org/tracks/rust/exercises/clock>

Warning

Important Exercise

Let's write a simple recursive evaluator for arithmetic expressions.

Complete the `eval` function in the file `session-2/tests/s2e2-evaluation.rs`.

Run tests with `cargo test --test s2e2-evaluation`.

1.	Quiz about last lecture	1
2.	Pattern matching	8
3.	Tooling intermezzo	25
3.1.	Debugger	25
3.2.	Clippy	26
3.3.	Summary	28
4.	Methods and traits	29
5.	Generics	48
6.	Closures	69
7.	Standard library types	85
8.	Conclusion	111

mod.rs - nu-lint - Visual Studio Code

RUN AND DEBUG | Debug unit tests | ignore_good.rs .../error_make_metadata | x.rs .../error_make_metadata | mod.rs .../error_make_metadata

src > rules > error_make_metadata > mod.rs > check

```
194 fn check_error_make_call(call: &Call, context: &LintContext) {
195     call.is_call_to_command(command_name: "error_make", context)
196     .then_some(()) Option<()>
197     .and_then(|()| call.get_first_positional_arg()) Option<&Expression>
198     .and_then(extract_record_from_expr) Option<&Vec<RecordItem>>
199     .and_then(|record: &Vec<RecordItem>| check_error_make_metadata(record, context,
200     call.span()))
201 }
202
203 1 reference
204 fn check(context: &LintContext) -> Vec<RuleViolation> {
205     context.collect_rule_violations(collector: |expr: &Expression, ctx: &
206     LintContext<'>| match &expr.expr {
207         Expr::Call(call: &Box<Call>) => check_error_make_call(call, context: ctx).
208         into_iter().collect(),
209         _ => vec![],
210     })
211 }
```

7: tid=741466 "rules:... PAUSED ON BREAKPOINT

nu_lint::rules::error_make_metadata
nu_lint::rule::Rule::assert_detect
nu_lint::context::LintContext::tes
nu_lint::rule::Rule::assert_detect

BREAKPOINTS
[x] Rust: on panic
[x] mod.rs src/rules/error_make_met... 203

MODULES
EXCLUDED CALLERS

PROBLEMS 4 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | Filter | Debug

nu-lint | organise-rules* | 4 0 | Debug unit tests in library 'nu_lint' (nu-lint) | Live Share | rust-analyzer | Format: auto | Disasm: auto | Deref: on | Console: cmd | nu-lint | Rust | Prettier



Clippy Lints Total number: 795

Lint levels 4 ▾

Lint groups 2 ▾

Version 0 ▾

Filter: Keywords or search string (^S` or `/` to focus) Clear

Applicability 4 ▾

All

Default

None

☐ Cargo

☒ Complexity

☒ Correctness

☐ Nursery

☐ Pedantic

☐ Perf

☐ Restriction

☐ Style

☐ Suspicious

☐ Deprecated

3.2. Clippy

Most useful lints:

- complexity
- style
- correctness

Place clippy rules in your `Cargo.toml` like this:

```
1 [lints.clippy]
2 complexity = { level = "deny", priority = -1 }
3 pedantic = { level = "deny", priority = -1 }
4 style = { level = "deny", priority = -1 }
5
6 absolute_paths = "deny"
7 allow_attributes_without_reason = "deny"
8 ...
```

(playground link)

Find more lint rules at rust-lang.github.io/rust-clippy.

3.3. Summary

Typical development workflow in Rust:

1. Debug with GDB / LLDB or VS Code
2. Format with `cargo fmt`
3. Lint with `cargo clippy`
4. Test with `cargo test`
5. Build with `cargo build` or `cargo build --release`

1. Quiz about last lecture	1
2. Pattern matching	8
3. Tooling intermezzo	25
4. Methods and traits	29
4.1. Methods	31
4.2. Receivers	34
4.3. Traits	35
4.4. Implementing traits	36
4.5. Practice: Rustlings Traits	38
4.6. Contracts	38
4.7. Super-traits	40
4.8. Associated Types	41
4.9. Deriving	44
4.10. Practice: Exercism Robot Simulator	46
4.11. Exercise: Logger trait	47
5. Generics	48
6. Closures	69
7. Standard library types	85

8. Conclusion	111
---------------------	-----

4.1. Methods

In Rust, methods are separated from fields:

```
1  #[derive(Debug)]
2  struct CarRace {
3      name: String,
4      laps: Vec<i32>,
5  }
6
7  impl CarRace {
8      // No receiver, a static method
9      fn new(name: &str) -> Self {
10         Self { name: String::from(name), laps: Vec::new() }
11     }
12     // Exclusive borrowed read-write access to self
13     fn add_lap(&mut self, lap: i32) {
14         self.laps.push(lap);
15     }
16 }
17
18 fn main() {
19     let mut race = CarRace::new("Monaco Grand Prix");
20     race.add_lap(70);
21 }
```

(playground link)

4.1. Methods

Like in other languages, methods group functionality around *acting* data types, types that *do* related things.

How are methods in Rust different from methods in other languages?

4.1. Methods

Like in other languages, methods group functionality around *acting* data types, types that *do* related things.

How are methods in Rust different from methods in other languages?

Methods cannot be overridden.

```
1 impl CarRace {  
2     fn add_lap(&mut self, lap: i32) {  
3         self.laps.push(lap);  
4     }  
5 }
```

(playground link)

What is the `&mut self` parameter syntax sugar for?

4.1. Methods

Like in other languages, methods group functionality around *acting* data types, types that *do* related things.

How are methods in Rust different from methods in other languages?

Methods cannot be overridden.

```
1 impl CarRace {  
2     fn add_lap(&mut self, lap: i32) {  
3         self.laps.push(lap);  
4     }  
5 }
```

[\(playground link\)](#)

What is the `&mut self` parameter syntax sugar for?

For `self: &mut Self`.

```
1 impl CarRace {  
2     fn add_lap(self: &mut Self, lap: i32) {  
3         self.laps.push(lap);  
4     }  
5 }
```

[\(playground link\)](#)

Methods can **consume** self:

```
1  #[derive(Debug)]
2  struct CarRace {
3      name: String,
4      laps: Vec<i32>,
5  }
6
7  impl CarRace {
8      // Exclusive ownership of self (covered later)
9      fn finish(self) {
10         let total: i32 = self.laps.iter().sum();
11         println!("Race {} is finished, total lap time:
12         {}", self.name, total);
13     }
14 }
15 fn main() {
16     let mut race = CarRace::new("Monaco Grand Prix");
17     race.add_lap(70);
18     race.finish();
19     // race.add_lap(42);
20 }
```

(playground link)

4.1. Methods

Methods can **consume** self:

```
1  #[derive(Debug)]
2  struct CarRace {
3      name: String,
4      laps: Vec<i32>,
5  }
6
7  impl CarRace {
8      // Exclusive ownership of self (covered later)
9      fn finish(self) {
10         let total: i32 = self.laps.iter().sum();
11         println!("Race {} is finished, total lap time:
12         {}", self.name, total);
13     }
14 }
15 fn main() {
16     let mut race = CarRace::new("Monaco Grand Prix");
17     race.add_lap(70);
18     race.finish();
19     // race.add_lap(42);
20 }
```

(playground link)

What is the name given to the `self` parameter in methods?

4.1. Methods

Methods can **consume** self:

```
1  #[derive(Debug)]
2  struct CarRace {
3      name: String,
4      laps: Vec<i32>,
5  }
6
7  impl CarRace {
8      // Exclusive ownership of self (covered later)
9      fn finish(self) {
10         let total: i32 = self.laps.iter().sum();
11         println!("Race {} is finished, total lap time:
12         {}", self.name, total);
13     }
14 }
15 fn main() {
16     let mut race = CarRace::new("Monaco Grand Prix");
17     race.add_lap(70);
18     race.finish();
19     // race.add_lap(42);
20 }
```

(playground link)

What is the name given to the `self` parameter in methods?

`self` is called the **receiver**.

After calling a method with `object.finish(self)`, you can no longer use `object`. It has been *consumed*.

Warning

Use the `self` receiver to define destructors or functionality that should happen only once at the end.

4.2. Receivers

Rust allows receivers to be one of the following:

- `self`, consuming `self`
- any kind of reference to `self`: `&self`, `&mut self`
- exceptions (covered later)

Name a few reference types in Rust.

4.2. Receivers

Rust allows receivers to be one of the following:

- `self`, consuming `self`
- any kind of reference to `self`: `&self`, `&mut self`
- exceptions (covered later)

Name a few reference types in Rust.

`&T`, `&mut T`, `Box<T>`, `Rc<T>`, `Arc<T>`, ...

Info

Reference types are also called **wrappers** and some are **smart pointers**.

Rust lets you abstract over types with traits. Similar to interfaces:

```
1 trait Pet {  
2     /// Return a sentence from this pet.  
3     fn talk(&self) -> String;  
4  
5     /// Print a string to the terminal greeting this pet.  
6     fn greet(&self);  
7 }
```

(playground link)

Properties of traits:

Rust lets you abstract over types with traits. Similar to interfaces:

```
1 trait Pet {  
2     /// Return a sentence from this pet.  
3     fn talk(&self) -> String;  
4  
5     /// Print a string to the terminal greeting this pet.  
6     fn greet(&self);  
7 }
```

(playground link)

Properties of traits:

- A trait is a list of methods that a type **must implement**

Rust lets you abstract over types with traits. Similar to interfaces:

```
1 trait Pet {  
2     /// Return a sentence from this pet.  
3     fn talk(&self) -> String;  
4  
5     /// Print a string to the terminal greeting this pet.  
6     fn greet(&self);  
7 }
```

(playground link)

Properties of traits:

- A trait is a list of methods that a type **must implement**
- The **signatures** of the type's methods **must be identical** to the trait's method signatures

Rust lets you abstract over types with traits. Similar to interfaces:

```
1 trait Pet {  
2     /// Return a sentence from this pet.  
3     fn talk(&self) -> String;  
4  
5     /// Print a string to the terminal greeting this pet.  
6     fn greet(&self);  
7 }
```

(playground link)

Properties of traits:

- A trait is a list of methods that a type **must implement**
- The **signatures** of the type's methods **must be identical** to the trait's method signatures

Info

Default implementations can be provided

4.4. Implementing traits

```
1  trait Pet {  
2      fn talk(&self) -> String;  
3  
4      fn greet(&self) {  
5          println!("Oh you're a cutie!  
6              What's your name? {}",  
7              self.talk());  
8      }  
9  }  
10  
11 struct Dog {  
12     name: String,  
13     age: i8,  
14 }  
15                                     (playground link)
```

4.4. Implementing traits

```
1  trait Pet {
2      fn talk(&self) -> String;
3
4      fn greet(&self) {
5          println!("Oh you're a cutie!
6              What's your name? {}",
7              self.talk());
8      }
9  }
10
11 struct Dog {
12     name: String,
13     age: i8,
14 }
```

(playground link)

```
1  impl Pet for Dog {
2      fn talk(&self) -> String {
3          format!("Woof, my name is {}", self.name)
4      }
5  }
6
7  fn main() {
8      let fido = Dog { name: String::from("Fido"),
9          age: 5 };
10     dbg!(fido.talk());
11     fido.greet();
12 }
```

(playground link)

4.4. Implementing traits

```
1  trait Pet {
2      fn talk(&self) -> String;
3
4      fn greet(&self) {
5          println!("Oh you're a cutie!
6              What's your name? {}",
7              self.talk());
8      }
9  }
10
11 struct Dog {
12     name: String,
13     age: i8,
14 }
```

(playground link)

```
1  impl Pet for Dog {
2      fn talk(&self) -> String {
3          format!("Woof, my name is {}", self.name)
4      }
5  }
6
7  fn main() {
8      let fido = Dog { name: String::from("Fido"),
9          age: 5 };
10     dbg!(fido.talk());
11     fido.greet();
12 }
```

(playground link)

Warning

a Cat type with a talk() method would not automatically satisfy Pet unless it is in an impl Pet block

4.4. Implementing traits

You can:

- split trait implementation blocks
- override default method implementations

```
1  impl Pet for Dog {  
2      fn talk(&self) -> String {  
3          format!("Woof, my name is {}!", self.name)  
4      }  
5  }  
6  
7  impl Pet for Dog {  
8      fn greet(&self) -> String {  
9          format!("Woof, my name is {}!", self.name)  
10     }  
11 }
```

(playground link)

Rustlings exercises:

- `exercises/15_traits/traits1.rs` - Basic trait implementation
- `exercises/15_traits/traits2.rs` - Trait methods
- `exercises/15_traits/traits3.rs` - Default implementations
- `exercises/15_traits/traits4.rs` - Trait bounds
- `exercises/15_traits/traits5.rs` - Advanced trait usage

vec
write
writeln

Keywords

SelfTy
as
async
await
become
break
const
continue
crate
dyn
else
enum
extern
false
fn

level of pointer indirection each time a new object is added to the mix (and, practically, a heap allocation).

Although there were other reasons as well, this issue of expensive composition is the key thing that drove Rust towards adopting a different model. It is particularly a problem when one considers, for example, the implications of composing together the `Future`s which will eventually make up an asynchronous task (including address-sensitive `async fn` state machines). It is plausible that there could be many layers of `Futures` composed together, including multiple layers of `async fns` handling different parts of a task. It was deemed unacceptable to force indirection and allocation for each layer of composition in this case.

`Pin<Ptr>` is an implementation of the third option. It allows us to solve the issues discussed with the second option by building a *shared contractual language* around the guarantees of “pinning” data.

Using `Pin<Ptr>` to pin values

In order to pin a value, we wrap a *pointer to that value* (of some type `Ptr`) in a `Pin<Ptr>`. `Pin<Ptr>` can wrap any pointer type, forming a promise that the **pointee** will not be *moved* or *otherwise invalidated*.

We call such a `Pin`-wrapped pointer a **pinning pointer**, (or pinning reference, or pinning `Box`, etc.) because its existence is the thing that is conceptually pinning the underlying pointee in place: it is the metaphorical “pin” securing the data in place on the pinboard (in memory).

Notice that the thing wrapped by `Pin` is not the value which we want to pin itself, but rather a pointer to that value! A `Pin<Ptr>` does not pin the `Ptr`; instead, it pins the pointer’s *pointee value*.

Pinning as a library contract

Pinning does not require nor make use of any compiler “magic”², only a specific contract between the `unsafe` parts of a library API and its users.

It is important to stress this point as a user of the `unsafe` parts of the `Pin` API. Practically, this means that performing the

Rust, being a systems programming language, focuses on safety.

4.6. Contracts

Rust, being a systems programming language, focuses on safety.

In Rust documentation, you will often encounter the word **contract**.

Info

A contract is an agreement between two parties. There are mainly two kinds of contracts in Rust:

- Contracts between **two pieces of code**
- Contracts between **the programmer and the compiler**

4.6. Contracts

Rust, being a systems programming language, focuses on safety.

In Rust documentation, you will often encounter the word **contract**.

Info

A contract is an agreement between two parties. There are mainly two kinds of contracts in Rust:

- Contracts between **two pieces of code**
- Contracts between **the programmer and the compiler**

Contracts between two pieces of code are generally safe (can be enforced automatically).

4.6. Contracts

Rust, being a systems programming language, focuses on safety.

In Rust documentation, you will often encounter the word **contract**.

Info

A contract is an agreement between two parties. There are mainly two kinds of contracts in Rust:

- Contracts between **two pieces of code**
- Contracts between **the programmer and the compiler**

Contracts between two pieces of code are generally safe (can be enforced automatically).

Warning

... but contracts between the programmer and the compiler can be unsafe or **may have to be checked by the programmer**.

Examples of such contracts: Pin, Send, Sync traits (advanced topic).

4.7. Super-traits

```
1  trait Animal {
2      fn leg_count(&self) -> u32;
3  }
4
5  trait Pet: Animal {
6      fn name(&self) -> String;
7  }
8
9  struct Dog(String);
10
11 impl Animal for Dog {
12     fn leg_count(&self) -> u32 {
13         4
14     }
15 }
16
17 impl Pet for Dog {
18     fn name(&self) -> String {
19         self.0.clone()
20     }
21 }
```

(playground link)

Super-traits are an extra constraint on traits that say: “to implement this trait, you must also implement that other trait”.

4.7. Super-traits

```
1  trait Animal {
2      fn leg_count(&self) -> u32;
3  }
4
5  trait Pet: Animal {
6      fn name(&self) -> String;
7  }
8
9  struct Dog(String);
10
11 impl Animal for Dog {
12     fn leg_count(&self) -> u32 {
13         4
14     }
15 }
16
17 impl Pet for Dog {
18     fn name(&self) -> String {
19         self.0.clone()
20     }
21 }
```

(playground link)

Super-traits are an extra constraint on traits that say: “to implement this trait, you must also implement that other trait”.

Warning

Super traits are the kind of language feature you should **avoid as long as you are stuck with the OOP mindset**.

Once you are willing to forget OOP, you can see super-traits are actually easy.

4.7. Super-traits

```
1  trait Animal {
2      fn leg_count(&self) -> u32;
3  }
4
5  trait Pet: Animal {
6      fn name(&self) -> String;
7  }
8
9  struct Dog(String);
10
11 impl Animal for Dog {
12     fn leg_count(&self) -> u32 {
13         4
14     }
15 }
16
17 impl Pet for Dog {
18     fn name(&self) -> String {
19         self.0.clone()
20     }
21 }
```

(playground link)

Super-traits are an extra constraint on traits that say: “to implement this trait, you must also implement that other trait”.

Warning

Super traits are the kind of language feature you should **avoid as long as you are stuck with the OOP mindset**.

Once you are willing to forget OOP, you can see super-traits are actually easy.

Advanced

... at least as long as you don't constrain **associated types** of super traits in subtraits

4.8. Associated Types

```
1  #[derive(Debug)]
2  struct Meters(i32);
3  #[derive(Debug)]
4  struct MetersSquared(i32);
5
6  trait Multiply {
7      type Output;
8      fn multiply(&self, other: &Self) -> Self::Output;
9  }
10
11 impl Multiply for Meters {
12     type Output = MetersSquared;
13     fn multiply(&self, other: &Self) -> Self::Output {
14         MetersSquared(self.0 * other.0)
15     }
16 }
17
18 fn main() {
19     println!("{:?}", Meters(10).multiply(&Meters(20)));
20 }
```

(playground link)

Associated types are **placeholder types** that are supplied by the trait implementation.

4.8. Associated Types

```
1  #[derive(Debug)]
2  struct Meters(i32);
3  #[derive(Debug)]
4  struct MetersSquared(i32);
5
6  trait Multiply {
7      type Output;
8      fn multiply(&self, other: &Self) -> Self::Output;
9  }
10
11 impl Multiply for Meters {
12     type Output = MetersSquared;
13     fn multiply(&self, other: &Self) -> Self::Output {
14         MetersSquared(self.0 * other.0)
15     }
16 }
17
18 fn main() {
19     println!("{:?}", Meters(10).multiply(&Meters(20)));
20 }
```

(playground link)

Associated types are **placeholder types** that are supplied by the trait implementation.

Why are associated types are sometimes also called “output types”

4.8. Associated Types

```
1  #[derive(Debug)]
2  struct Meters(i32);
3  #[derive(Debug)]
4  struct MetersSquared(i32);
5
6  trait Multiply {
7      type Output;
8      fn multiply(&self, other: &Self) -> Self::Output;
9  }
10
11 impl Multiply for Meters {
12     type Output = MetersSquared;
13     fn multiply(&self, other: &Self) -> Self::Output {
14         MetersSquared(self.0 * other.0)
15     }
16 }
17
18 fn main() {
19     println!("{:?}", Meters(10).multiply(&Meters(20)));
20 }
```

(playground link)

Associated types are **placeholder types** that are supplied by the trait implementation.

Why are associated types sometimes also called “output types”

The implementer, not the caller, chooses the concrete associated type.

4.8. Associated Types

```

1  #[derive(Debug)]
2  struct Meters(i32);
3  #[derive(Debug)]
4  struct MetersSquared(i32);
5
6  trait Multiply {
7      type Output;
8      fn multiply(&self, other: &Self) -> Self::Output;
9  }
10
11 impl Multiply for Meters {
12     type Output = MetersSquared;
13     fn multiply(&self, other: &Self) -> Self::Output {
14         MetersSquared(self.0 * other.0)
15     }
16 }
17
18 fn main() {
19     println!("{:?}", Meters(10).multiply(&Meters(20)));
20 }

```

(playground link)

Associated types are **placeholder types** that are supplied by the trait implementation.

Why are associated types are sometimes also called “output types”

The implementer, not the caller, chooses the concrete associated type.

Iterators from the standard library have an associated type Item:

```

1  pub trait Iterator {
2      type Item;
3      fn next(&mut self) ->
        Option<Self::Item>;
4  }

```

(playground link)

4.8. Associated Types

Use cases for associated types: implementing
(network) **protocols**.

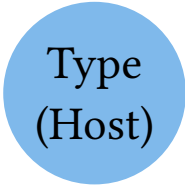
4.8. Associated Types

```
1  trait Protocol {
2      type Message;
3      fn send(&self, msg: Self::Message);
4      fn receive(&self) -> Self::Message;
5  }
6
7  impl Protocol for Tcp {
8      type Message = TcpPacket;
9      fn send(&self, msg: Self::Message)
10         {
11         // Send the TCP packet
12     }
13     fn receive(&self) -> Self::Message
14     {
15         // Receive a TCP packet
16     }
17 }
```

(playground link)

4.8. Associated Types

Use cases for associated types: implementing
(network) **protocols**.



Type
(Host)

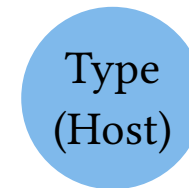
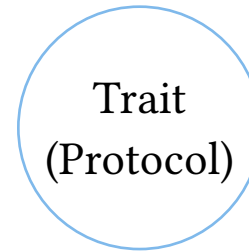
4.8. Associated Types

```
1  trait Protocol {
2      type Message;
3      fn send(&self, msg: Self::Message);
4      fn receive(&self) -> Self::Message;
5  }
6
7  impl Protocol for Tcp {
8      type Message = TcpPacket;
9      fn send(&self, msg: Self::Message)
10         {
11         // Send the TCP packet
12         }
13      fn receive(&self) -> Self::Message
14         {
15         // Receive a TCP packet
16         }
17  }
```

(playground link)

4.8. Associated Types

Use cases for associated types: implementing (network) **protocols**.

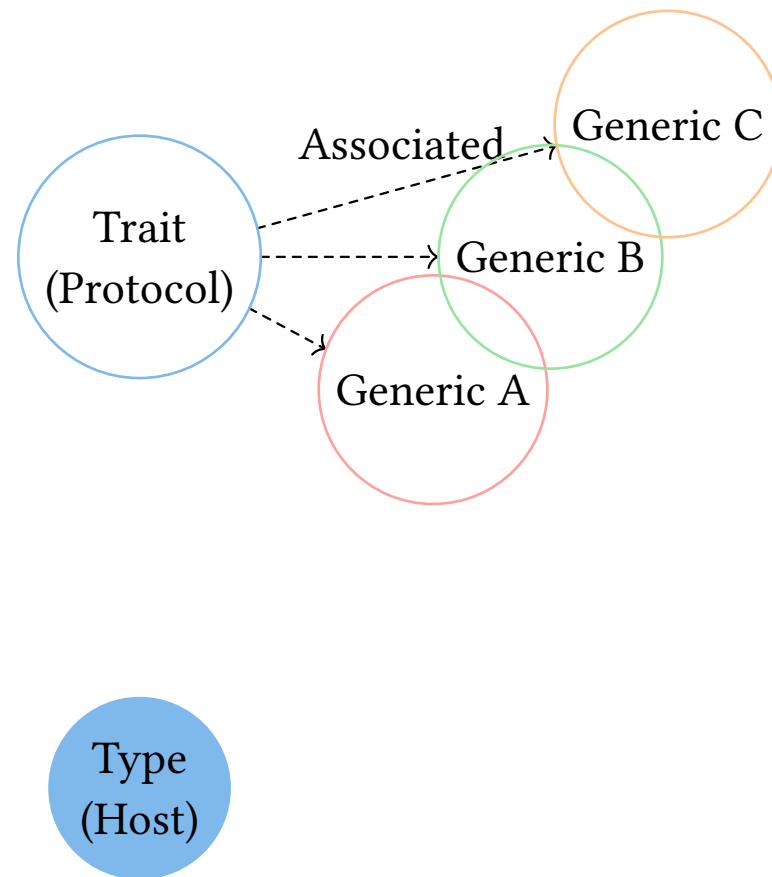


```
1  trait Protocol {
2      type Message;
3      fn send(&self, msg: Self::Message);
4      fn receive(&self) -> Self::Message;
5  }
6
7  impl Protocol for Tcp {
8      type Message = TcpPacket;
9      fn send(&self, msg: Self::Message)
10         {
11         // Send the TCP packet
12     }
13     fn receive(&self) -> Self::Message
14     {
15         // Receive a TCP packet
16     }
17 }
```

(playground link)

4.8. Associated Types

Use cases for associated types: implementing (network) **protocols**.



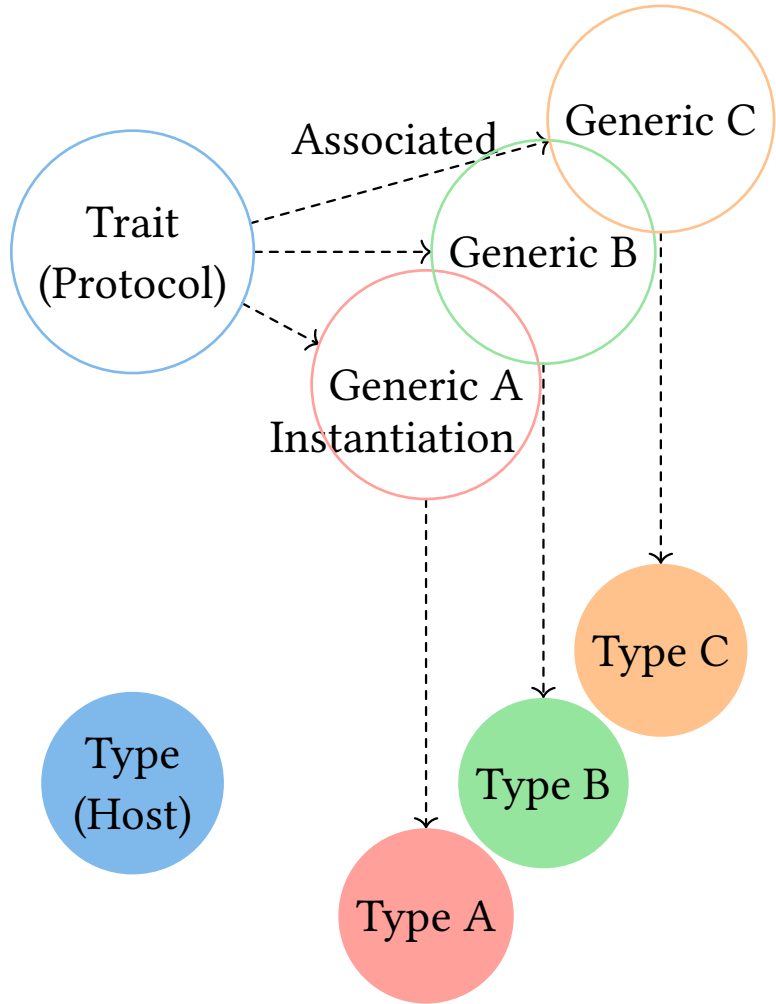
4.8. Associated Types

```
1  trait Protocol {
2      type Message;
3      fn send(&self, msg: Self::Message);
4      fn receive(&self) -> Self::Message;
5  }
6
7  impl Protocol for Tcp {
8      type Message = TcpPacket;
9      fn send(&self, msg: Self::Message)
10         {
11         // Send the TCP packet
12     }
13     fn receive(&self) -> Self::Message
14     {
15         // Receive a TCP packet
16     }
17 }
```

(playground link)

4.8. Associated Types

Use cases for associated types: implementing (network) **protocols**.

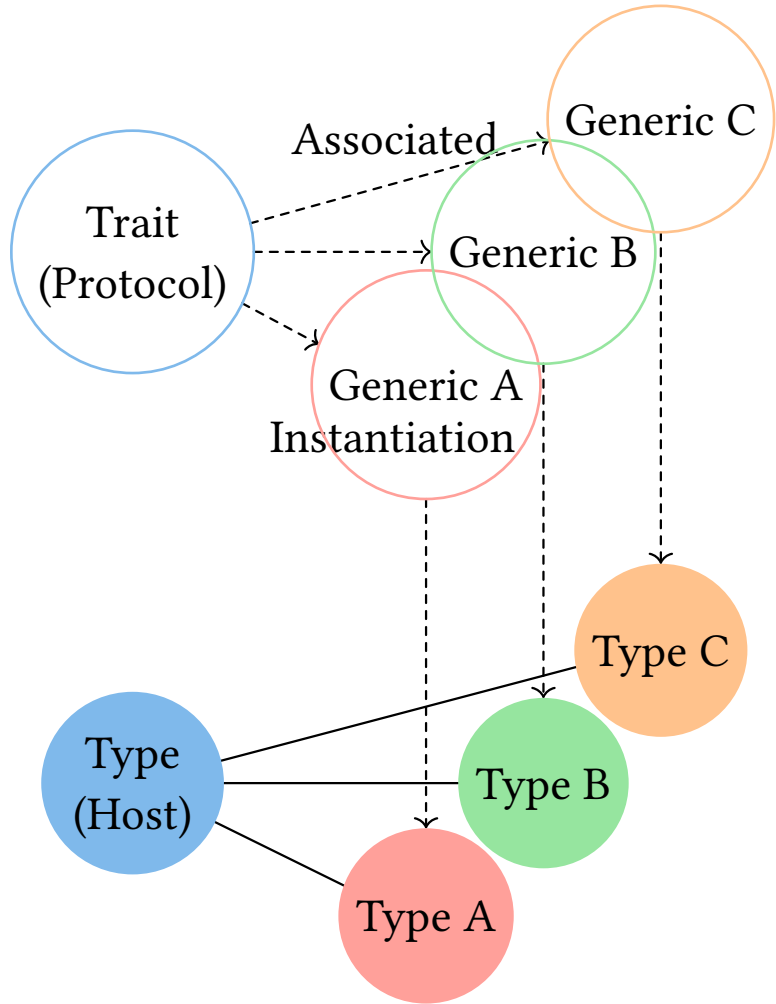


```
1  trait Protocol {
2      type Message;
3      fn send(&self, msg: Self::Message);
4      fn receive(&self) -> Self::Message;
5  }
6
7  impl Protocol for Tcp {
8      type Message = TcpPacket;
9      fn send(&self, msg: Self::Message)
10         {
11         // Send the TCP packet
12     }
13     fn receive(&self) -> Self::Message
14     {
15         // Receive a TCP packet
16     }
17 }
```

(playground link)

4.8. Associated Types

Use cases for associated types: implementing (network) **protocols**.

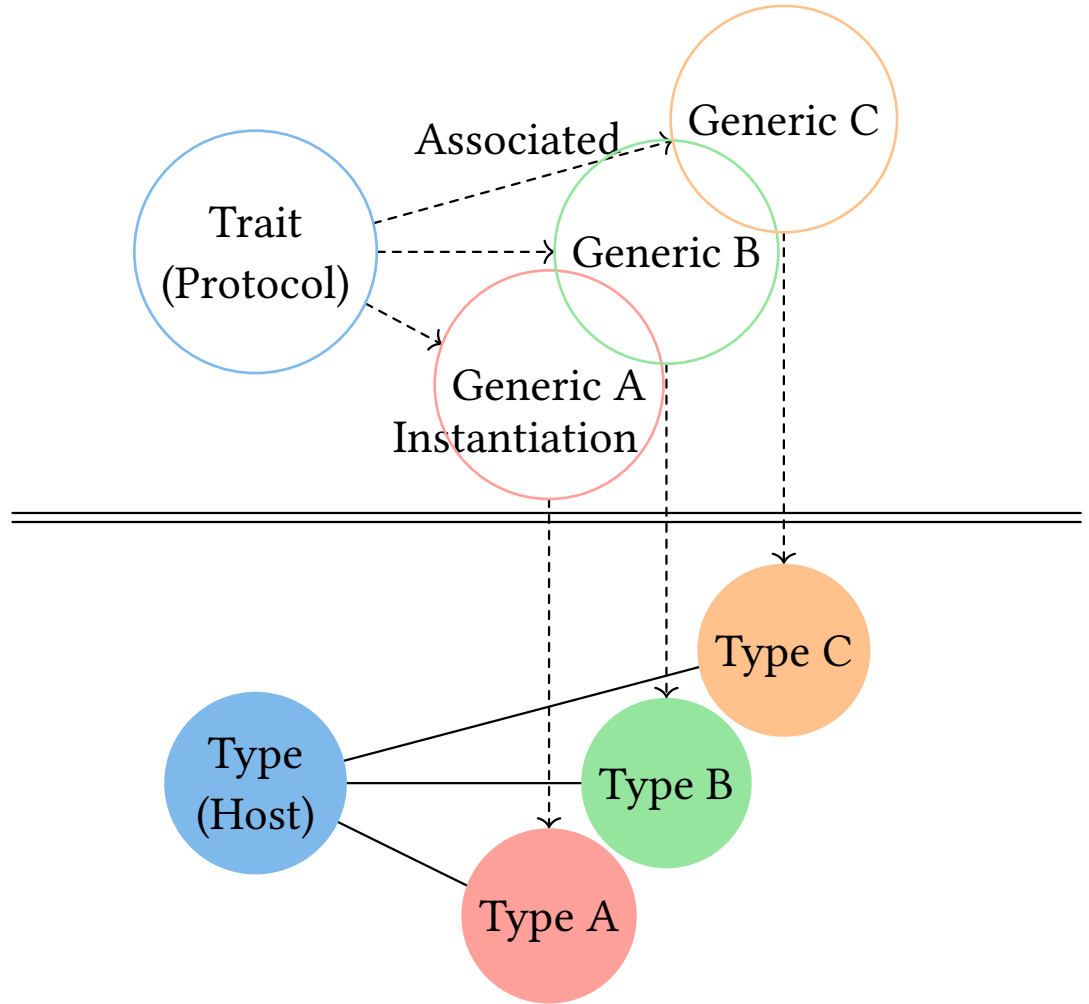


```
1  trait Protocol {
2      type Message;
3      fn send(&self, msg: Self::Message);
4      fn receive(&self) -> Self::Message;
5  }
6
7  impl Protocol for Tcp {
8      type Message = TcpPacket;
9      fn send(&self, msg: Self::Message)
10         {
11         // Send the TCP packet
12     }
13     fn receive(&self) -> Self::Message
14     {
15         // Receive a TCP packet
16     }
17 }
```

(playground link)

4.8. Associated Types

Use cases for associated types: implementing (network) **protocols**.



4.8. Associated Types

```
1  trait Protocol {
2      type Message;
3      fn send(&self, msg: Self::Message);
4      fn receive(&self) -> Self::Message;
5  }
6
7  impl Protocol for Tcp {
8      type Message = TcpPacket;
9      fn send(&self, msg: Self::Message)
10         {
11         // Send the TCP packet
12     }
13     fn receive(&self) -> Self::Message
14     {
15         // Receive a TCP packet
16     }
17 }
```

(playground link)

4.9. Deriving

Supported traits can be automatically implemented for your custom types, as follows:

```
1  #[derive(Debug, Clone, Default)]
2  struct Player {
3      name: String,
4      strength: u8,
5      hit_points: u8,
6  }
7
8  fn main() {
9      let p1 = Player::default(); // Default trait adds `default` constructor.
10     let mut p2 = p1.clone(); // Clone trait adds `clone` method.
11     p2.name = String::from("EldurScrollz");
12     // Debug trait adds support for printing with `{:?}`.
13     println!("{p1:?} vs. {p2:?}");
14 }
```

([playground link](#))

How is the derive functionality implemented in Rust?

4.9. Deriving

Supported traits can be automatically implemented for your custom types, as follows:

```
1  #[derive(Debug, Clone, Default)]
2  struct Player {
3      name: String,
4      strength: u8,
5      hit_points: u8,
6  }
7
8  fn main() {
9      let p1 = Player::default(); // Default trait adds `default` constructor.
10     let mut p2 = p1.clone(); // Clone trait adds `clone` method.
11     p2.name = String::from("EldurScrollz");
12     // Debug trait adds support for printing with `{:?}`.
13     println!("{p1:?} vs. {p2:?}");
14 }
```

(playground link)

How is the derive functionality implemented in Rust?

With **procedural macros**, and many crates provide useful derive macros to add useful functionality.

4.9. Deriving

Supported traits can be automatically implemented for your custom types, as follows:

```
1  #[derive(Debug, Clone, Default)]
2  struct Player {
3      name: String,
4      strength: u8,
5      hit_points: u8,
6  }
7
8  fn main() {
9      let p1 = Player::default(); // Default trait adds `default` constructor.
10     let mut p2 = p1.clone(); // Clone trait adds `clone` method.
11     p2.name = String::from("EldurScrollz");
12     // Debug trait adds support for printing with `{:?}`.
13     println!("{p1:?} vs. {p2:?}");
14 }
```

(playground link)

How is the derive functionality implemented in Rust?

With **procedural macros**, and many crates provide useful derive macros to add useful functionality.

For example, `serde` can derive serialization support for a struct using `#[derive(Serialize)]`.

4.9. Deriving

4.9.1. Why is deriving useful?

A manual implementation of the Clone trait for the Player struct would look like this:

```
1 impl Clone for Player {  
2     fn clone(&self) -> Self {  
3         Player {  
4             name: self.name.clone(),  
5             strength: self.strength.clone(),  
6             hit_points: self.hit_points.clone(),  
7         }  
8     }  
9 }
```

(playground link)

It is easier to just write `#[derive(Clone)]`

Info

The derive attribute is similar to deriving in Haskell.

Exercism: Robot Simulator:

- Path: `exercises/practice/robot-simulator/`
- Goal: Simulate a robot moving on a grid
- Requires: Structs, enums (Direction), methods, pattern matching

Online: <https://exercism.org/tracks/rust/exercises/robot-simulator>

4.11. Exercise: Logger trait

Warning**Important Exercise**

Complete the test code in `session-2/examples/s2e3-logger.rs`.

Run code with `cargo run --example s2e3-logger`.

1. Quiz about last lecture	1
2. Pattern matching	8
3. Tooling intermezzo	25
4. Methods and traits	29
5. Generics	48
5.1. Generic functions	50
5.2. Monomorphisation	51
5.3. Trait bounds	52
5.4. Practice: Rustlings Generics	53
5.5. Combining traits	54
5.6. Composition over inheritance	55
5.7. where clauses	56
5.8. Feature of where clauses	57
5.9. Generic datatypes	58
5.10. Generic traits	62
5.11. Associated types vs. generic type parameters	64
5.12. Multiple generic impl blocks	65
5.13. impl Trait	66

5.14. Exercise	68
6. Closures	69
7. Standard library types	85
8. Conclusion	111

5.1. Generic functions

```
1 fn pick<T>(cond: bool, left: T, right: T) -> T {  
2     if cond { left } else { right }  
3 }  
4  
5 fn main() {  
6     println!("picked a number: {:?}", pick(true, 222, 333));  
7     println!("picked a string: {:?}", pick(false, 'L', 'R'));  
8 }
```

(playground link)

Bodies of generic functions need to be well-defined for all possible types T.

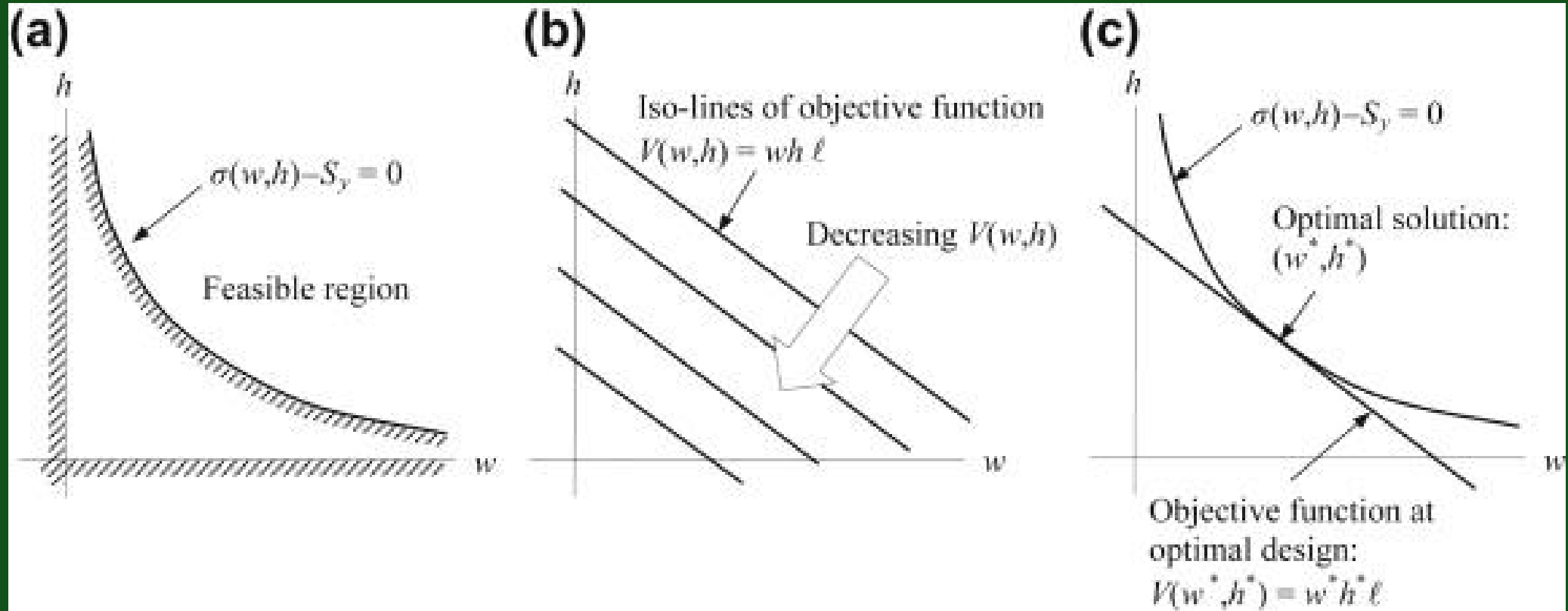
5.2. Monomorphisation

The compiler **generates concrete versions** of generic functions for each used type.

```
1 fn pick_i32(cond: bool, left: i32, right: i32) -> i32 {  
2     if cond { left } else { right }  
3 }  
4  
5 fn pick_char(cond: bool, left: char, right: char) -> char {  
6     if cond { left } else { right }  
7 }
```

(playground link)

Generics are a zero-cost abstraction.



5.3. Trait bounds

Require the types to implement some trait, so that you can call this trait's methods

```
1  fn duplicate<T: Clone>(a: T) -> (T, T) {  
2      (a.clone(), a.clone())  
3  }  
4  
5  struct NotCloneable;  
6  
7  fn main() {  
8      let foo = String::from("foo");  
9      let pair = duplicate(foo);  
10     println!("{pair:?}");  
11 }
```

(playground link)

What happens if we pass NotCloneable to duplicate? (Try in playground!)

5.3. Trait bounds

Require the types to implement some trait, so that you can call this trait's methods

```
1  fn duplicate<T: Clone>(a: T) -> (T, T) {  
2      (a.clone(), a.clone())  
3  }  
4  
5  struct NotCloneable;  
6  
7  fn main() {  
8      let foo = String::from("foo");  
9      let pair = duplicate(foo);  
10     println!("{pair:?}");  
11 }
```

(playground link)

What happens if we pass NotCloneable to duplicate? (Try in playground!)

A compile-time error, because NotCloneable does not implement the Clone trait.

5.4. Practice: Rustlings Generics

Rustlings exercises:

- `exercises/14_generics/generics1.rs` - Basic generic functions
- `exercises/14_generics/generics2.rs` - Generic implementations

5.5. Combining traits

When multiple traits are necessary, use + to join them.

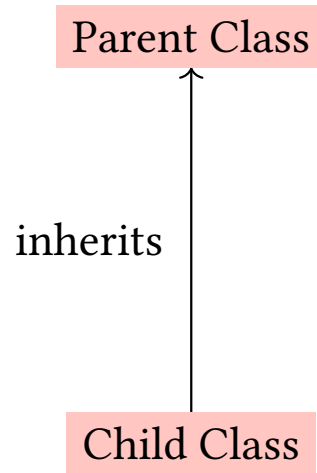
```
1 fn compare_and_print<T: PartialOrd + Display>(a: T, b: T) {  
2     if a < b {  
3         println!("{}", a, b);  
4     } else {  
5         println!("{}", a, b);  
6     }  
7 }
```

(playground link)

5.6. Composition over inheritance

Rust forbids object inheritance completely.

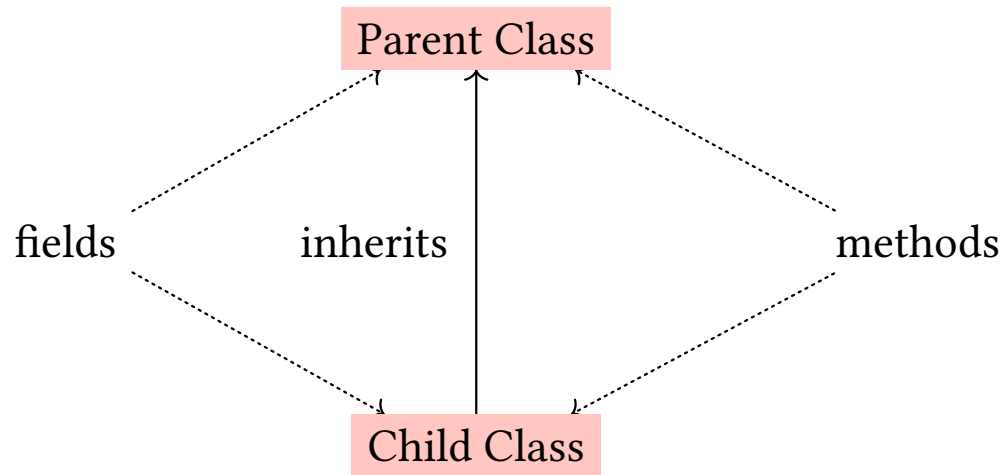
Class inheritance



5.6. Composition over inheritance

Rust forbids object inheritance completely.

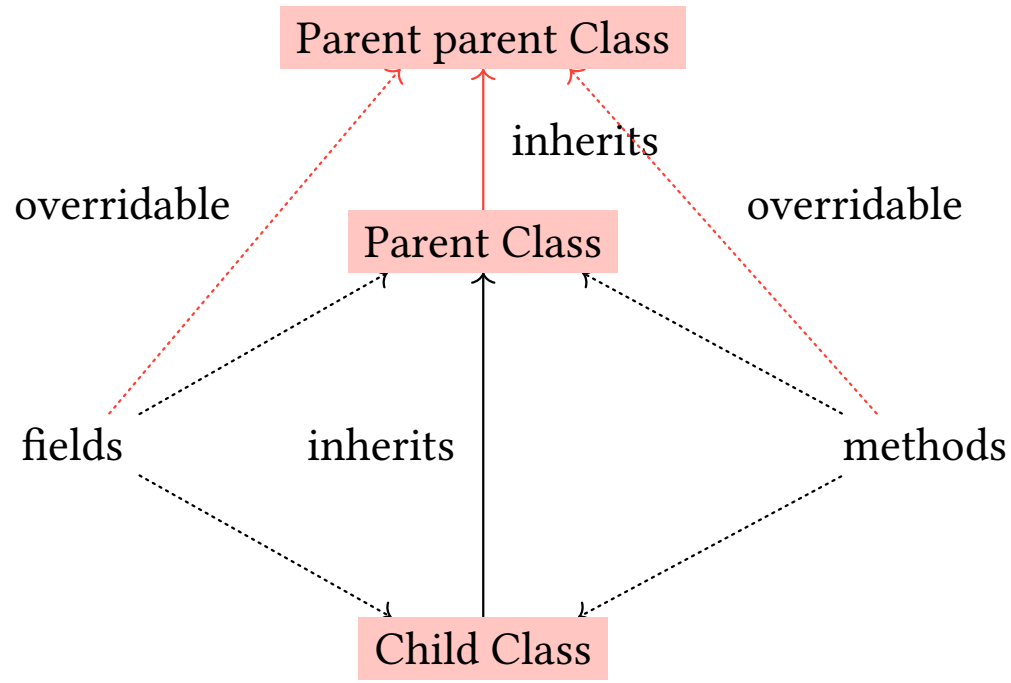
Class inheritance



5.6. Composition over inheritance

Rust forbids object inheritance completely.

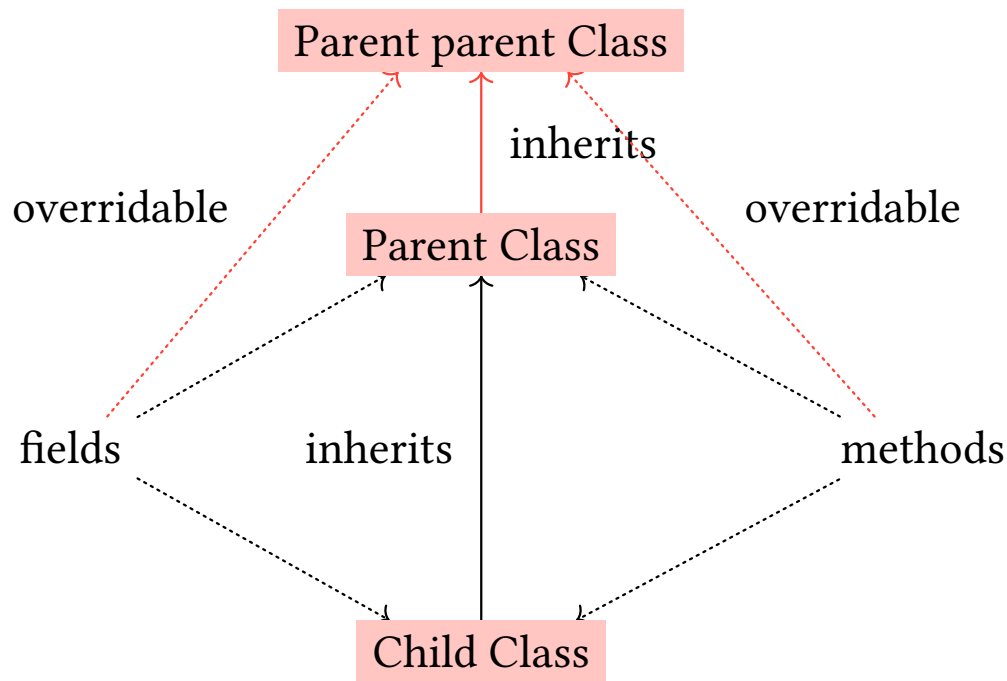
Class inheritance



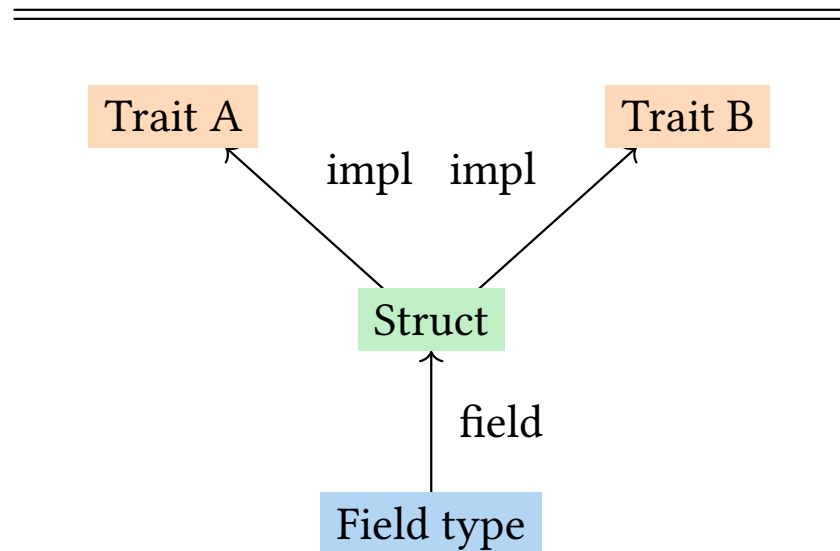
5.6. Composition over inheritance

Rust forbids object inheritance completely.

Class inheritance



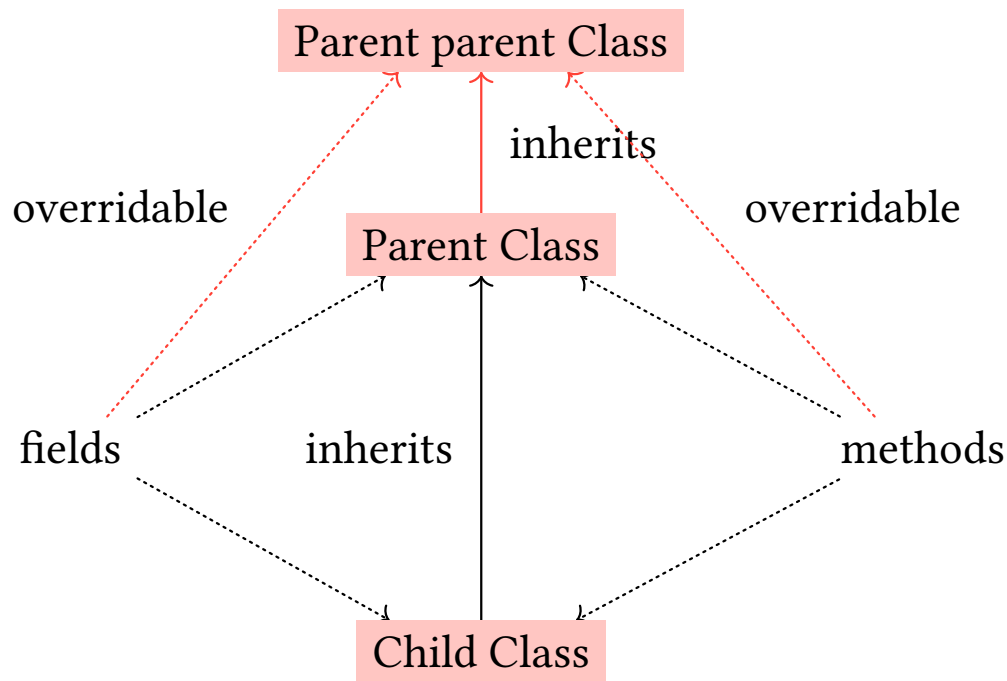
Composition with traits



5.6. Composition over inheritance

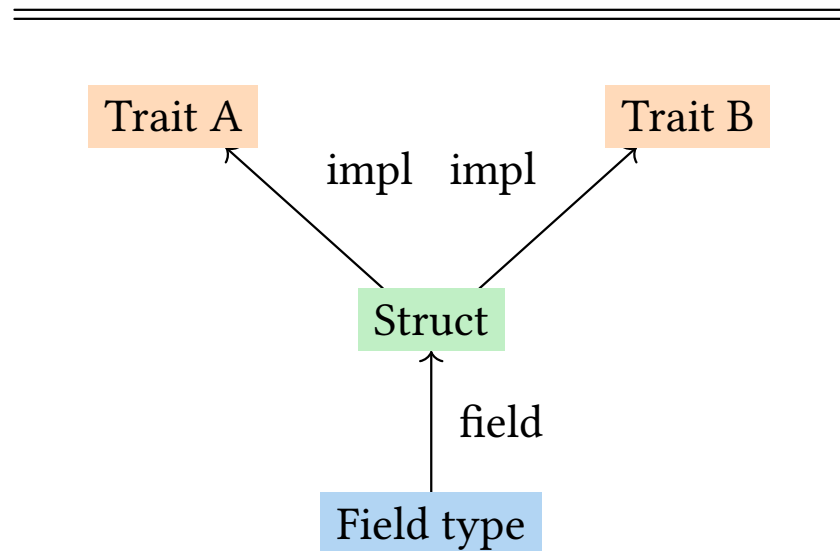
Rust forbids object inheritance completely.

Class inheritance



Are traits in Rust a kind of multiple inheritance?

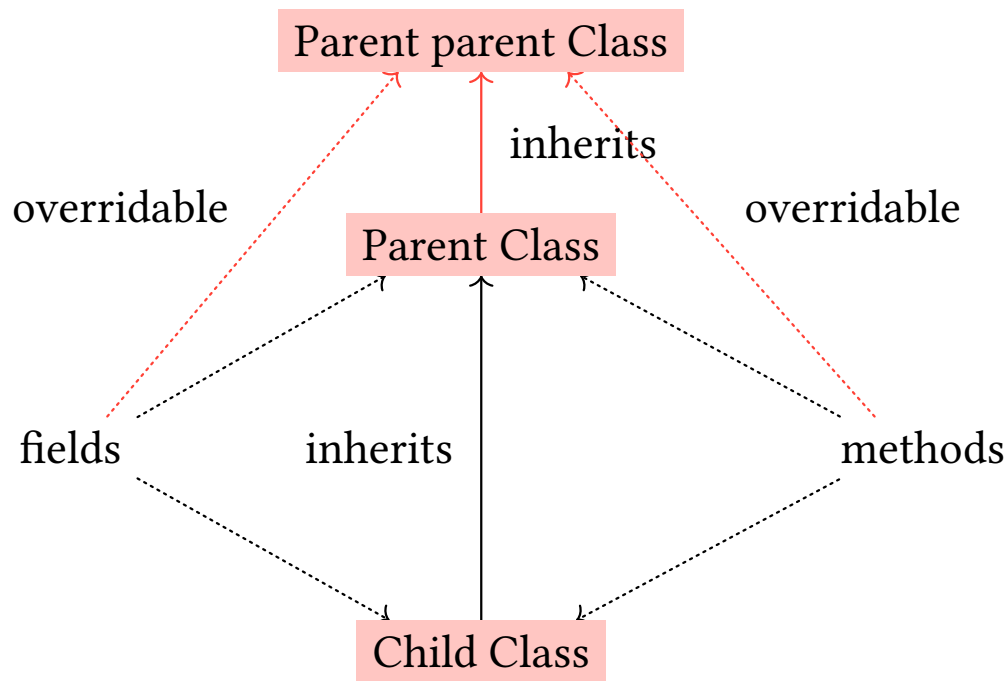
Composition with traits



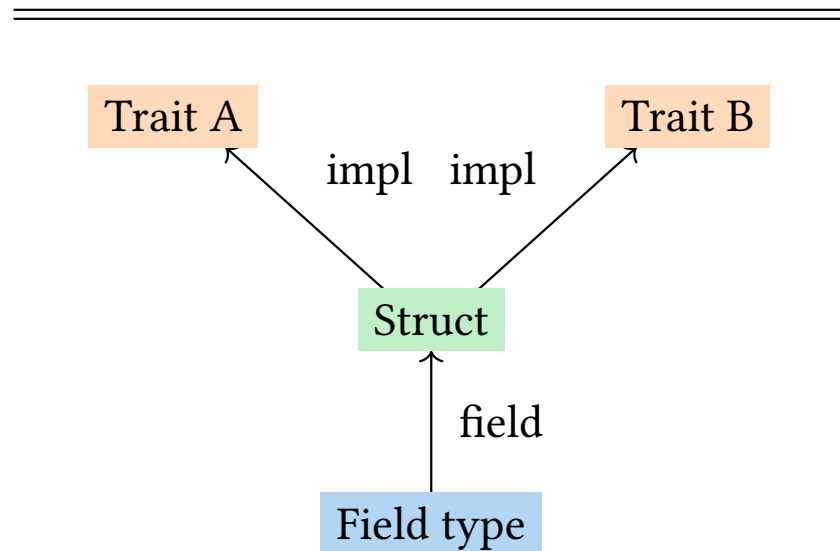
5.6. Composition over inheritance

Rust forbids object inheritance completely.

Class inheritance



Composition with traits



Are traits in Rust a kind of multiple inheritance?

Yes, but no data inheritance. No diamond problem (see example diamond-problem).

5.7. where clauses

Declutters the function signature if you have many parameters

```
1 fn duplicate<T>(a: T) -> (T, T)
2 where
3     T: Clone,
4 {
5     (a.clone(), a.clone())
6 }
```

([playground link](#))

5.8. Feature of where clauses

With where clauses you can put **trait bounds on composite types**:

```
1  fn duplicate_option<T>(a: Option<T>) -> (Option<T>, Option<T>)
2  where
3      Option<T>: Clone,
4  {
5      (a.clone(), a.clone())
6  }
7
8  fn main() {
9      let s = Some(String::from("hello"));
10     let pair = duplicate_option(s);
11     println!("{pair:?}");
12 }
```

(playground link)

Info

Powerful feature of generic programming in Rust!

5.9. Generic datatypes

```
1  pub trait Logger {  
2      /// Log a message at the given verbosity level.  
3      fn log(&self, verbosity: u8, message: &str);  
4  }  
5  
6  struct StderrLogger;  
7  
8  impl Logger for StderrLogger {  
9      fn log(&self, verbosity: u8, message: &str) {  
10          eprintln!("verbosity={verbosity}: {message}");  
11      }  
12  } (playground link)
```

5.9. Generic datatypes

```

1 pub trait Logger {
2     /// Log a message at the given verbosity level.
3     fn log(&self, verbosity: u8, message: &str);
4 }
5
6 struct StderrLogger;
7
8 impl Logger for StderrLogger {
9     fn log(&self, verbosity: u8, message: &str) {
10         eprintln!("verbosity={verbosity}: {message}");
11     }
12 }

```

[\(playground link\)](#)

You can implement a trait for a generic datatype **if you provide template type parameters**.

Only log messages up to the given verbosity level:

```

1 struct VerbosityFilter<L> {
2     max_verbosity: u8,
3     inner: L,
4 }
5
6 impl<L: Logger> Logger for VerbosityFilter<L> {
7     fn log(&self, verbosity: u8, message: &str) {
8         if verbosity <= self.max_verbosity {
9             self.inner.log(verbosity, message);
10        }
11    }
12 }

```

[\(playground link\)](#)

Why is L specified twice in `impl<L: Logger> .. VerbosityFilter<L>`?
Isn't that redundant?

5.9. Generic datatypes

```

1  pub trait Logger {
2      /// Log a message at the given verbosity level.
3      fn log(&self, verbosity: u8, message: &str);
4  }
5
6  struct StderrLogger;
7
8  impl Logger for StderrLogger {
9      fn log(&self, verbosity: u8, message: &str) {
10         eprintln!("verbosity={verbosity}: {message}");
11     }
12 }

```

(playground link)

You can implement a trait for a generic datatype **if you provide template type parameters**.

Only log messages up to the given verbosity level:

```

1  struct VerbosityFilter<L> {
2      max_verbosity: u8,
3      inner: L,
4  }
5
6  impl<L: Logger> Logger for VerbosityFilter<L> {
7      fn log(&self, verbosity: u8, message: &str) {
8          if verbosity <= self.max_verbosity {
9              self.inner.log(verbosity, message);
10         }
11     }
12 }

```

(playground link)

Why is L specified twice in `impl<L: Logger> .. VerbosityFilter<L>{`?
Isn't that redundant?

`impl` parameters are separate and usually carries trait bounds (not the datatype).

What happens if you would just use a concrete type as in `impl VerbosityFilter<StderrLogger> { .. }`

5.9. Generic datatypes

```

1  pub trait Logger {
2      /// Log a message at the given verbosity level.
3      fn log(&self, verbosity: u8, message: &str);
4  }
5
6  struct StderrLogger;
7
8  impl Logger for StderrLogger {
9      fn log(&self, verbosity: u8, message: &str) {
10         eprintln!("verbosity={verbosity}: {message}");
11     }
12 }

```

[\(playground link\)](#)

You can implement a trait for a generic datatype **if you provide template type parameters**.

Only log messages up to the given verbosity level:

```

1  struct VerbosityFilter<L> {
2      max_verbosity: u8,
3      inner: L,
4  }
5
6  impl<L: Logger> Logger for VerbosityFilter<L> {
7      fn log(&self, verbosity: u8, message: &str) {
8          if verbosity <= self.max_verbosity {
9              self.inner.log(verbosity, message);
10         }
11     }
12 }

```

[\(playground link\)](#)

Why is L specified twice in `impl<L: Logger> .. VerbosityFilter<L>`?
Isn't that redundant?

`impl` parameters are separate and usually carries trait bounds (not the datatype).

What happens if you would just use a concrete type as in `impl VerbosityFilter<StderrLogger> { .. }`

Would only work with `StderrLogger` instances, not with any other type that implements `Logger`.

5.9. Generic datatypes

5.9.1. Blanket impl blocks

A blanket implementation is an impl block that applies to **all types** that satisfy the trait bounds.

```
1 struct VerbosityFilter<L> {  
2     max_verbosity: u8,  
3     inner: L,  
4 }  
5  
6 impl<L: Logger> Logger for VerbosityFilter<L> {  
7     fn log(&self, verbosity: u8, message: &str) {  
8         if verbosity <= self.max_verbosity {  
9             self.inner.log(verbosity, message);  
10        }  
11    }  
12 }
```

[\(playground link\)](#)

It is a bit like in mathematics:

$$\forall l \in \text{Logger} : \text{VerbosityFilter}(l) \rightarrow \text{Logger}(l)$$

5.9. Generic datatypes

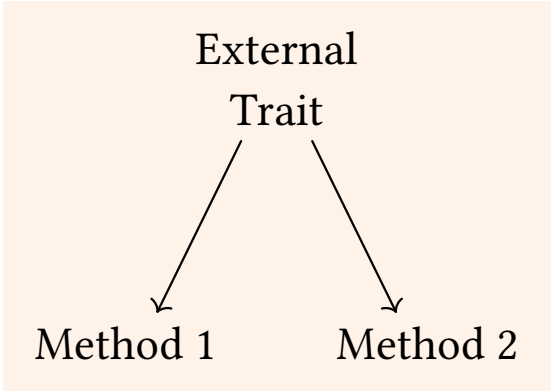
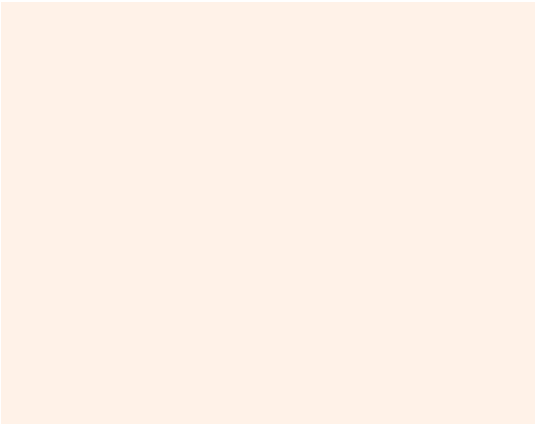
5.9.2. Visualisation

External
library

External
Trait

5.9.2. Visualisation

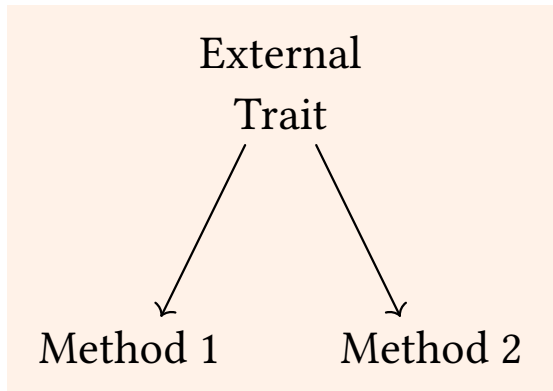
External
library



5.9. Generic datatypes

5.9.2. Visualisation

**External
library**

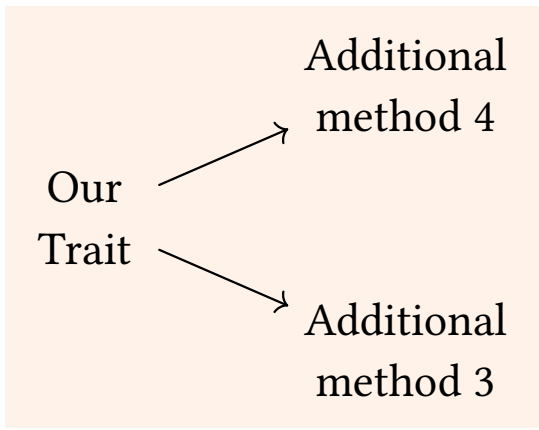
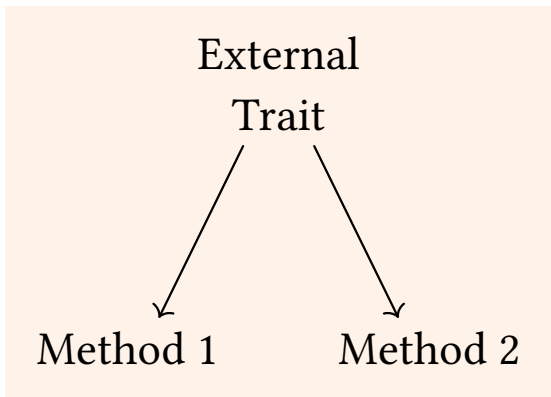


Our
Trait

Our concrete
type

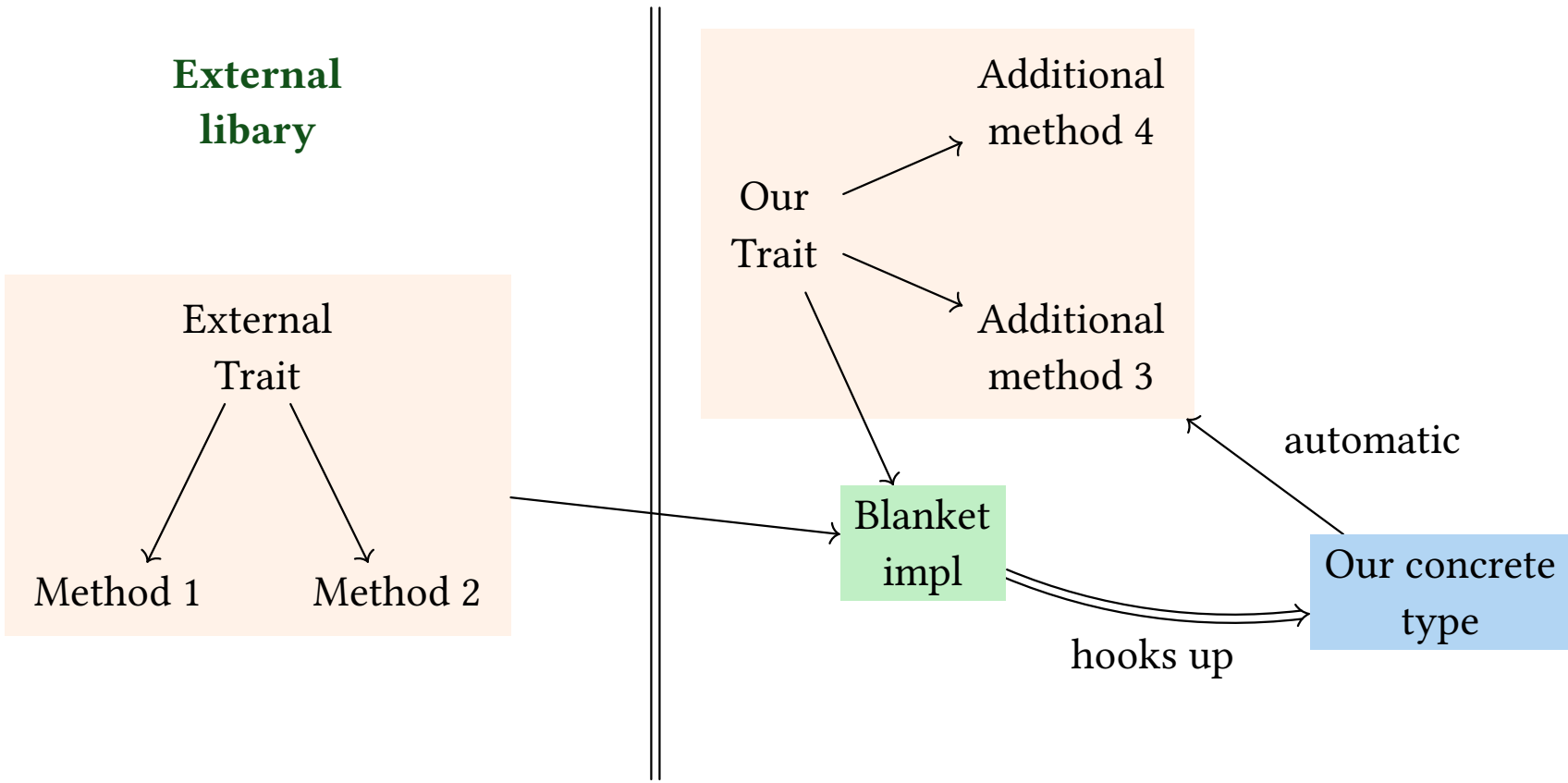
5.9.2. Visualisation

External
library



Our concrete
type

5.9.2. Visualisation



5.9. Generic datatypes

5.9.3. Putting constraints on super-traits (advanced)

A subtrait can add constraints on the associated types of its supertrait.

```

1  trait Container {
2      type Item;
3      fn get(&self) -> &Self::Item;
4  }
5
6  trait PrintableContainer: Container
7  where
8      Self::Item: Display,
9  {
10     fn print_item(&self) {
11         println!("Item: {}", self.get());
12     }
13 }
14
15 struct Box<T> {
16     value: T,
17 }
```

(playground link)

```

1  impl<T> Container for Box<T> {
2      type Item = T;
3      fn get(&self) -> &Self::Item {
4          &self.value
5      }
6  }
7
8  impl<T: Display> PrintableContainer for Box<T> {}
9
10 fn main() {
11     let b = Box { value: 42 };
12     b.print_item();
13 }
```

(playground link)

Why does PrintableContainer require Self::Item: Display?

5.9. Generic datatypes

5.9.3. Putting constraints on super-traits (advanced)

A subtrait can add constraints on the associated types of its supertrait.

```

1  trait Container {
2      type Item;
3      fn get(&self) -> &Self::Item;
4  }
5
6  trait PrintableContainer: Container
7  where
8      Self::Item: Display,
9  {
10     fn print_item(&self) {
11         println!("Item: {}", self.get());
12     }
13 }
14
15 struct Box<T> {
16     value: T,
17 }
```

(playground link)

```

1  impl<T> Container for Box<T> {
2      type Item = T;
3      fn get(&self) -> &Self::Item {
4          &self.value
5      }
6  }
7
8  impl<T: Display> PrintableContainer for Box<T> {}
9
10 fn main() {
11     let b = Box { value: 42 };
12     b.print_item();
13 }
```

(playground link)

Why does PrintableContainer require Self::Item: Display?

It constrains the supertrait's associated type so that print_item can use the Display trait.

5.10. Generic traits

The From trait is a standard library trait for type conversion.

```
1 pub trait From<T>: Sized {  
2     fn from(value: T) -> Self;  
3 }  
4  
5 #[derive(Debug)]  
6 struct Foo(String);
```

([playground link](#))

Info

Common traits in the standard library will be covered later on.

5.10. Generic traits

5.10.1. Example From

```
1  impl From<u32> for Foo {
2      fn from(from: u32) -> Foo {
3          Foo(format!("Converted from integer: {from}"))
4      }
5  }
6
7  impl From<bool> for Foo {
8      fn from(from: bool) -> Foo {
9          Foo(format!("Converted from bool: {from}"))
10     }
11 }
12 fn main() {
13     dbg!(Foo::from(123));
14     dbg!(Foo::from(true));
15 }
```

[\(playground link\)](#)

Warning

Notice that a **generic trait can be implemented multiple times** for the same type!

5.11. Associated types vs. generic type parameters

It may not always be clear when to pick an associated type or a generic type.

Associated types

(chosen by the implementer)

Generic
datatype

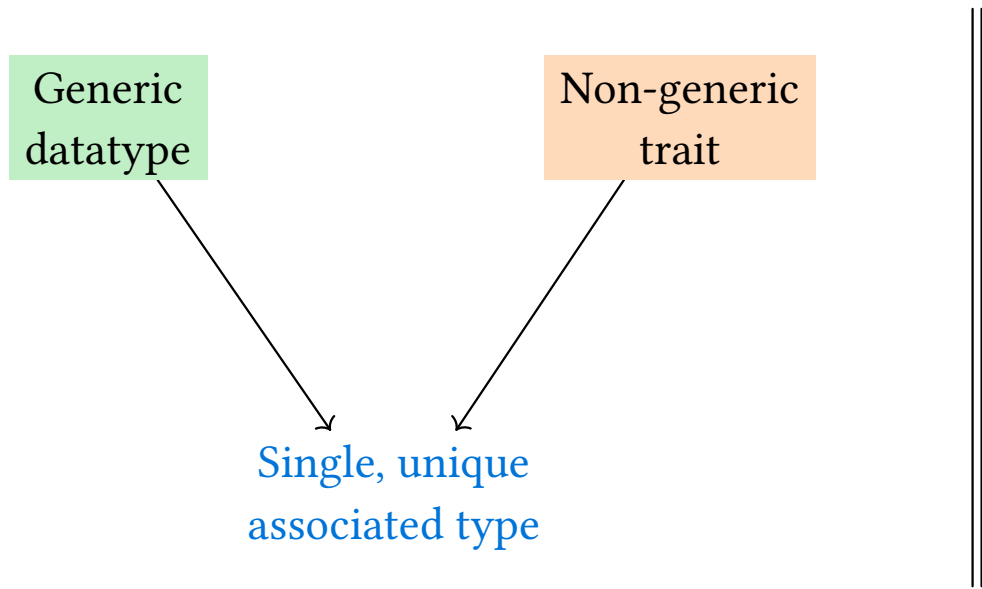
Non-generic
trait

5.11. Associated types vs. generic type parameters

It may not always be clear when to pick an associated type or a generic type parameter.

Associated types

(chosen by the implementer)

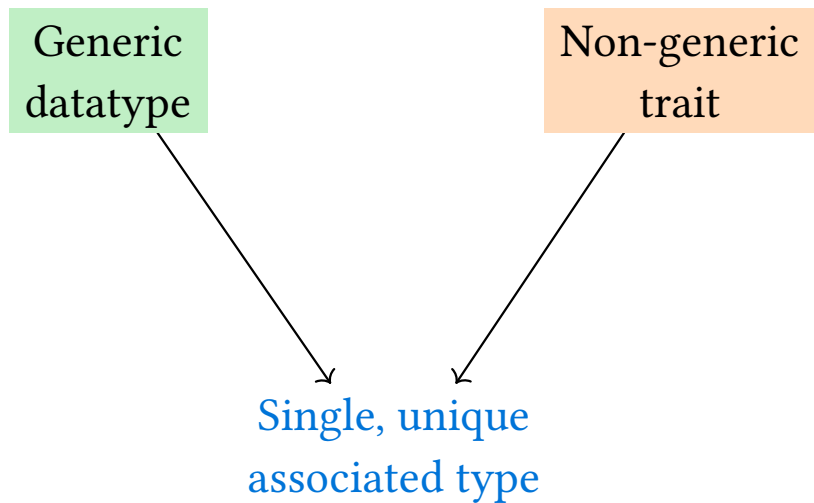


5.11. Associated types vs. generic type parameters

It may not always be clear when to pick an associated type or a generic type.

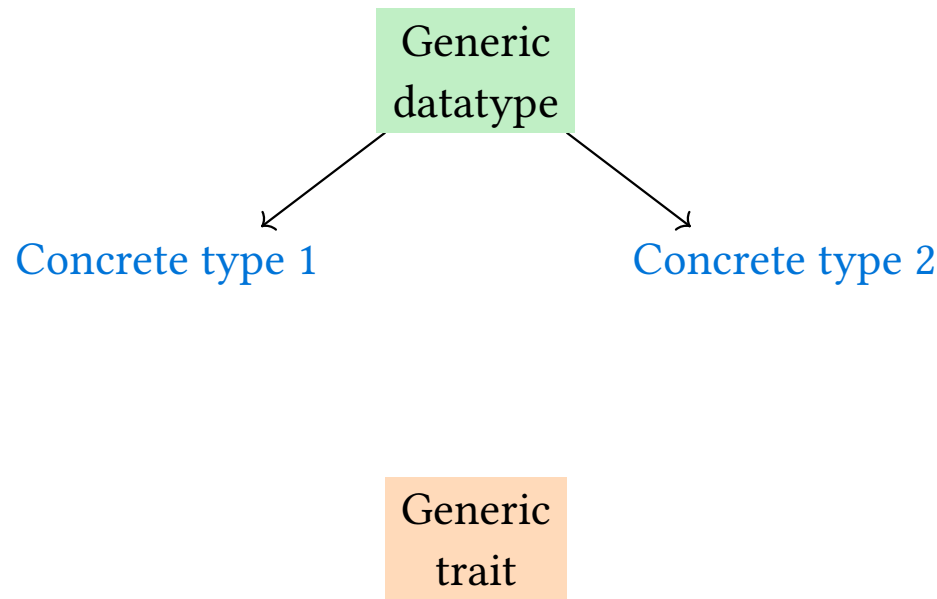
Associated types

(chosen by the implementer)



Generic types parameters

(chosen by the caller)

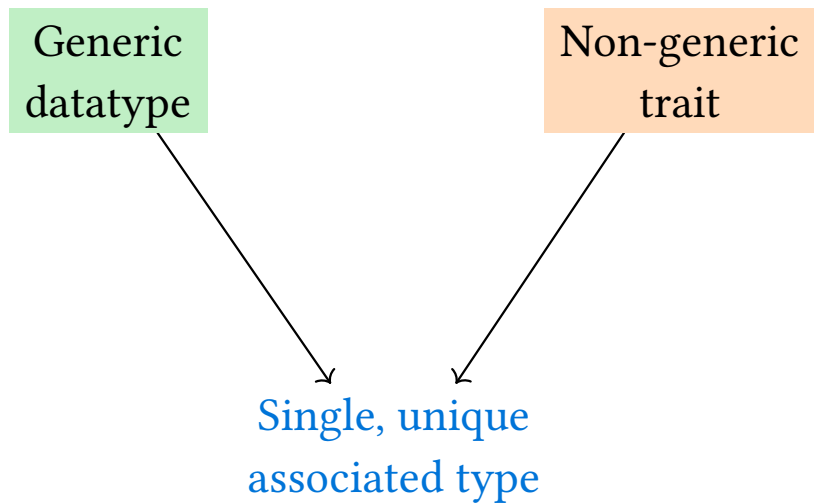


5.11. Associated types vs. generic type parameters

It may not always be clear when to pick an associated type or a generic type.

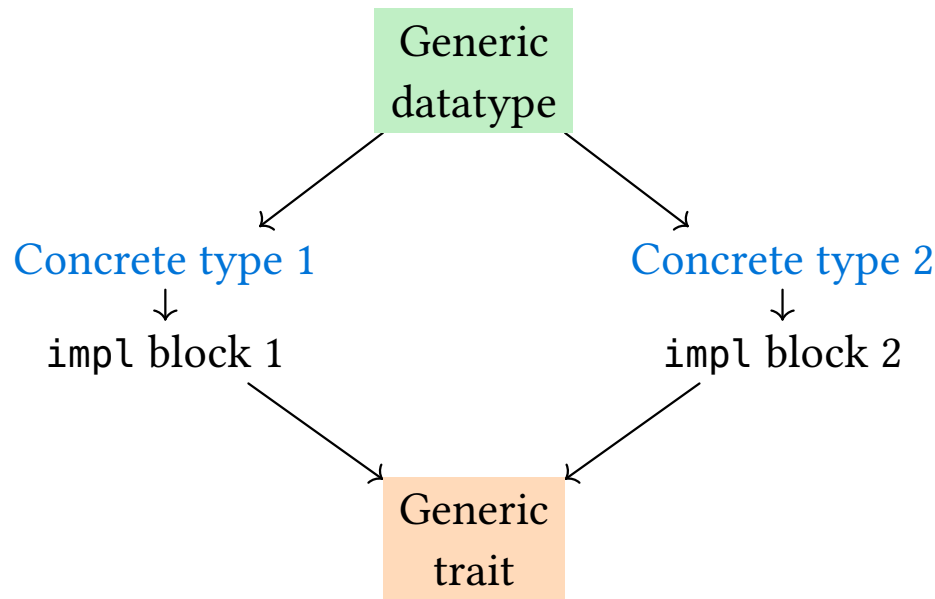
Associated types

(chosen by the implementer)



Generic types parameters

(chosen by the caller)

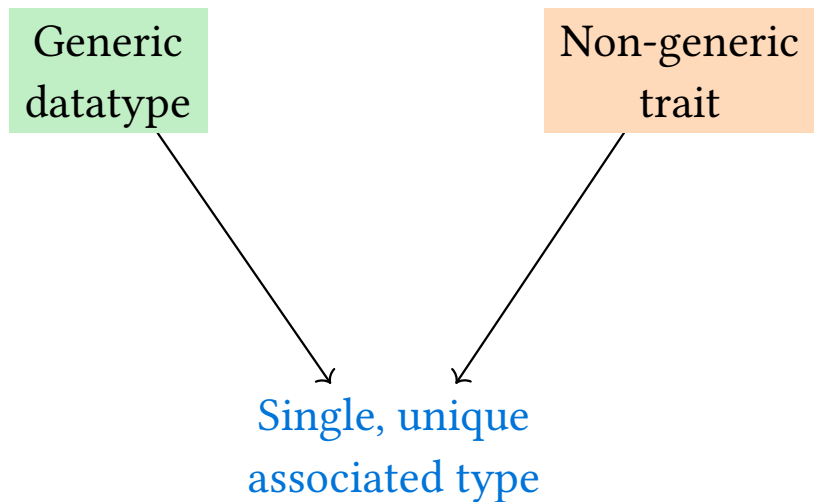


5.11. Associated types vs. generic type parameters

It may not always be clear when to pick an associated type or a generic type.

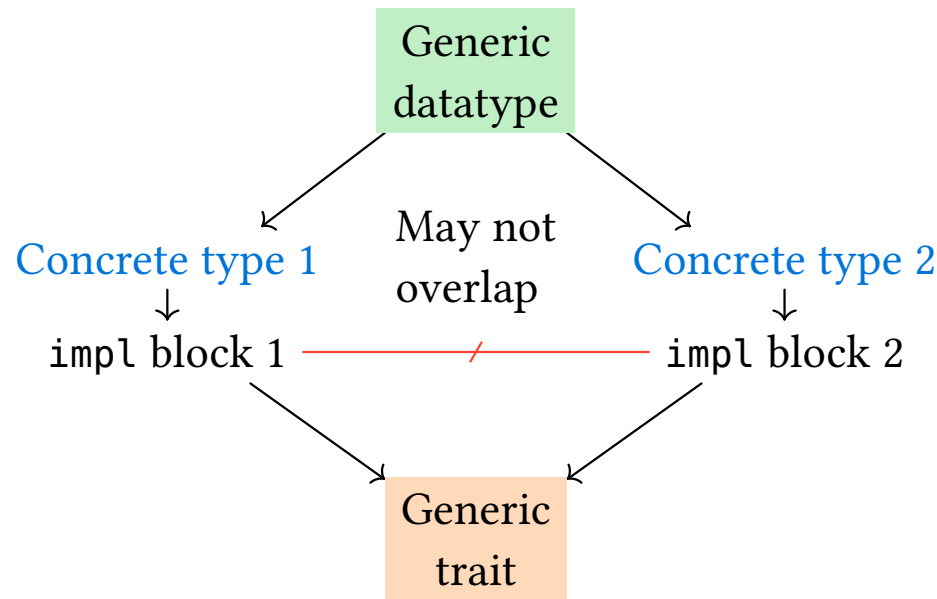
Associated types

(chosen by the implementer)



Generic types parameters

(chosen by the caller)



- Associated types behave like output (first choice).
- Generic type parameters behave like input.

5.12. Multiple generic impl blocks

```
1  struct Container<T> {
2      value: T,
3  }
4
5  impl<T> Container<T> {
6      fn process(&self) -> &'static str { "generic implementation" }
7  }
8
9  impl Container<i32> {
10     fn process(&self) -> &'static str { "specialized implementation for i32" }
11 }
```

(playground link)

Will this code compile?

5.12. Multiple generic impl blocks

```
1  struct Container<T> {  
2    value: T,  
3  }  
4  
5  impl<T> Container<T> {  
6    fn process(&self) -> &'static str { "generic implementation" }  
7  }  
8  
9  impl Container<i32> {  
10   fn process(&self) -> &'static str { "specialized implementation for i32" }  
11 }  
  
(playground link)
```

Will this code compile?

No. Rust does not allow overlapping implementations.

5.13. impl Trait

Info

`impl Into<i32>` is syntactic sugar for: `fn add_42_millions<T: Into<i32>>(x: T) -> i32.`
T is an anonymous and **hidden generic type**.

```
1  fn add_42_millions(x: impl Into<i32>) -> i32 {
2      x.into() + 42_000_000
3  }
4
5  fn pair_of(x: u32) -> impl Debug {
6      (x + 1, x - 1)
7  }
8
9  fn main() {
10     let many = add_42_millions(42_i8);
11     dbg!(many);
12     let many_more = add_42_millions(10_000_000);
13     dbg!(many_more);
14     let debuggable = pair_of(27);
15     dbg!(debuggable);
16 }
```

(playground link)

5.13. impl Trait

5.13.1. Inference for return type impl Trait

```
1  fn returns_impl_trait(x: i32) -> impl Display {  
2      if x > 0 {  
3          x  
4      } else {  
5          -x  
6      }  
7  }  
8  
9  fn main() {  
10     let result = returns_impl_trait(-5);  
11     println!("{}", result);  
12 }
```

(playground link)

What is the return type of `returns_impl_trait`?

5.13. impl Trait

5.13.1. Inference for return type impl Trait

```
1  fn returns_impl_trait(x: i32) -> impl Display {  
2      if x > 0 {  
3          x  
4      } else {  
5          -x  
6      }  
7  }  
8  
9  fn main() {  
10     let result = returns_impl_trait(-5);  
11     println!("{}", result);  
12 }
```

(playground link)

What is the return type of `returns_impl_trait`?

`i32`, because both branches return an `i32`.

Warning

Important Exercise

Please find the exercise in `./session-2/tests/s2e4-min.rs`.

Test using the command: `cargo test --test s2e4-min -- --nocapture`

1. Quiz about last lecture	1
2. Pattern matching	8
3. Tooling intermezzo	25
4. Methods and traits	29
5. Generics	48
6. Closures	69
6.1. Syntax	71
6.2. Capturing	72
6.3. Moving	74
6.4. Internal representation of closures	75
6.5. Datastructures containing functions	76
6.6. Practice: Rustlings Iterators	77
6.7. Practice: Exercism Accumulate	78
6.8. Closure traits	79
6.9. Picking the right closure trait	81
6.10. Auto-traits of closures	82
6.11. Exercise	84
7. Standard library types	85

8. Conclusion	111
---------------------	-----

Structure and Interpretation of Computer Programs

Second Edition



Harold Abelson and
Gerald Jay Sussman
with Julie Sussman

Closures

Anonymous functions that can capture variables from their surrounding scope.

```
1  fn main() {  
2    // Argument and return type can be inferred for lightweight syntax:  
3    let double_it = |n| n * 2;  
4    dbg!(double_it(50));  
5  
6    // Or we can specify types and bracket the body to be fully explicit:  
7    let add_1f32 = |x: f32| -> f32 { x + 1.0 };  
8    dbg!(add_1f32(50.));  
9  }
```

(playground link)

Closures

Anonymous functions that can capture variables from their surrounding scope.

```
1  fn main() {  
2    // Argument and return type can be inferred for lightweight syntax:  
3    let double_it = |n| n * 2;  
4    dbg!(double_it(50));  
5  
6    // Or we can specify types and bracket the body to be fully explicit:  
7    let add_1f32 = |x: f32| -> f32 { x + 1.0 };  
8    dbg!(add_1f32(50.));  
9  }
```

(playground link)

- The body may be be surrounded by { .. }

Closures

Anonymous functions that can capture variables from their surrounding scope.

```
1  fn main() {  
2    // Argument and return type can be inferred for lightweight syntax:  
3    let double_it = |n| n * 2;  
4    dbg!(double_it(50));  
5  
6    // Or we can specify types and bracket the body to be fully explicit:  
7    let add_1f32 = |x: f32| -> f32 { x + 1.0 };  
8    dbg!(add_1f32(50.));  
9  }
```

(playground link)

- The body may be surrounded by { .. }
- Argument and return types are optional, and are inferred if not given.

6.2. Capturing

6.2.1. Immutable capture

```
1  fn main() {  
2      let max_value = 5;  
3      let clamp = |v| {  
4          if v > max_value  
            { max_value } else { v }  
5      };  
6  
7      dbg!(clamp(1));  
8      dbg!(clamp(3));  
9      dbg!(clamp(5));  
10     dbg!(clamp(7));  
11 } (playground link)
```

By default, a closure captures values by immutable reference. Here `max_value` is captured by `clamp`, but still available to `main` for printing.

Try making `max_value` mutable, changing it, and printing the clamped values again. Why doesn't this work?

6.2. Capturing

6.2.1. Immutable capture

```
1  fn main() {  
2      let max_value = 5;  
3      let clamp = |v| {  
4          if v > max_value  
            { max_value } else { v }  
5      };  
6  
7      dbg!(clamp(1));  
8      dbg!(clamp(3));  
9      dbg!(clamp(5));  
10     dbg!(clamp(7));  
11 }
```

(playground link)

By default, a closure captures values by immutable reference. Here `max_value` is captured by `clamp`, but still available to `main` for printing.

Try making `max_value` mutable, changing it, and printing the clamped values again. Why doesn't this work?

The closure captures by reference. No mutable borrow may coexist with an immutable borrow.

6.2. Capturing

6.2.2. Mutable capture

```
1  fn main() {  
2      let mut max_value = 5;  
3      let mut clamp = |v| {  
4          max_value += 1;  
5          if v > max_value  
            { max_value } else { v }  
6      };  
7  
8      dbg!(clamp(1));  
9      dbg!(clamp(3));  
10     dbg!(clamp(5));  
11     dbg!(clamp(7));  
12     dbg!(clamp(10));  
13 } (playground link)
```

If a closure mutates captured values, it captures them by mutable reference.

6.2. Capturing

6.2.2. Mutable capture

```
1  fn main() {  
2      let mut max_value = 5;  
3      let mut clamp = |v| {  
4          max_value += 1;  
5          if v > max_value  
            { max_value } else { v }  
6      };  
7  
8      dbg!(clamp(1));  
9      dbg!(clamp(3));  
10     dbg!(clamp(5));  
11     dbg!(clamp(7));  
12     dbg!(clamp(10));  
13 }
```

(playground link)

If a closure mutates captured values, it captures them by mutable reference.

Why does the closure need to be mut?

6.2. Capturing

6.2.2. Mutable capture

```
1  fn main() {  
2      let mut max_value = 5;  
3      let mut clamp = |v| {  
4          max_value += 1;  
5          if v > max_value  
            { max_value } else { v }  
6      };  
7  
8      dbg!(clamp(1));  
9      dbg!(clamp(3));  
10     dbg!(clamp(5));  
11     dbg!(clamp(7));  
12     dbg!(clamp(10));  
13 }
```

(playground link)

If a closure mutates captured values, it captures them by mutable reference.

Why does the closure need to be mut?

The closure's internal state changes, so it must be declared mutable.

6.2. Capturing

6.2.2. Mutable capture

```
1  fn main() {  
2      let mut max_value = 5;  
3      let mut clamp = |v| {  
4          max_value += 1;  
5          if v > max_value  
            { max_value } else { v }  
6      };  
7  
8      dbg!(clamp(1));  
9      dbg!(clamp(3));  
10     dbg!(clamp(5));  
11     dbg!(clamp(7));  
12     dbg!(clamp(10));  
13 }
```

(playground link)

If a closure mutates captured values, it captures them by mutable reference.

Why does the closure need to be mut?

The closure's internal state changes, so it must be declared mutable.

What happens if you try to access `max_value` directly while the closure exists?

6.2. Capturing

6.2.2. Mutable capture

```
1  fn main() {  
2      let mut max_value = 5;  
3      let mut clamp = |v| {  
4          max_value += 1;  
5          if v > max_value  
            { max_value } else { v }  
6      };  
7  
8      dbg!(clamp(1));  
9      dbg!(clamp(3));  
10     dbg!(clamp(5));  
11     dbg!(clamp(7));  
12     dbg!(clamp(10));  
13 }
```

(playground link)

If a closure mutates captured values, it captures them by mutable reference.

Why does the closure need to be mut?

The closure's internal state changes, so it must be declared mutable.

What happens if you try to access `max_value` directly while the closure exists?

Compile error: the closure holds a mutable borrow, preventing other accesses.

6.3. Moving

6.3.1. Move closures

```
1  fn main() {  
2      let name =  
        String::from("Alice");  
3      let greet = move || {  
4          println!("Hello, {}", name);  
5      };  
6  
7      greet();  
8      greet();  
9  
10     // println!("{}", name); //  
        Error!  
11 }
```

(playground link)

The `move` keyword forces a closure to take ownership of captured values.

6.3. Moving

6.3.1. Move closures

```
1  fn main() {  
2      let name =  
        String::from("Alice");  
3      let greet = move || {  
4          println!("Hello, {}", name);  
5      };  
6  
7      greet();  
8      greet();  
9  
10     // println!("{}", name); //  
        Error!  
11 }
```

(playground link)

The `move` keyword forces a closure to take ownership of captured values.

What happens if you uncomment the last line?

6.3. Moving

6.3.1. Move closures

```
1  fn main() {  
2      let name =  
        String::from("Alice");  
3      let greet = move || {  
4          println!("Hello, {}", name);  
5      };  
6  
7      greet();  
8      greet();  
9  
10     // println!("{}", name); //  
        Error!  
11 }
```

(playground link)

The move keyword forces a closure to take ownership of captured values.

What happens if you uncomment the last line?

Compile error: name was moved into the closure and can no longer be used.

6.3. Moving

6.3.1. Move closures

```
1  fn main() {  
2      let name =  
        String::from("Alice");  
3      let greet = move || {  
4          println!("Hello, {}", name);  
5      };  
6  
7      greet();  
8      greet();  
9  
10     // println!("{}", name); //  
        Error!  
11 }
```

(playground link)

The move keyword forces a closure to take ownership of captured values.

What happens if you uncomment the last line?

Compile error: name was moved into the closure and can no longer be used.

Why can we call greet() multiple times?

6.3. Moving

6.3.1. Move closures

```
1  fn main() {  
2      let name =  
        String::from("Alice");  
3      let greet = move || {  
4          println!("Hello, {}", name);  
5      };  
6  
7      greet();  
8      greet();  
9  
10     // println!("{}", name); //  
        Error!  
11 }
```

(playground link)

The `move` keyword forces a closure to take ownership of captured values.

What happens if you uncomment the last line?

Compile error: `name` was moved into the closure and can no longer be used.

Why can we call `greet()` multiple times?

The closure owns `name`, which implements `Clone`, so it can be called repeatedly.

6.3. Moving

6.3.1. Move closures

```
1  fn main() {  
2      let name =  
        String::from("Alice");  
3      let greet = move || {  
4          println!("Hello, {}", name);  
5      };  
6  
7      greet();  
8      greet();  
9  
10     // println!("{}", name); //  
        Error!  
11 }
```

(playground link)

The move keyword forces a closure to take ownership of captured values.

What happens if you uncomment the last line?

Compile error: name was moved into the closure and can no longer be used.

Why can we call `greet()` multiple times?

The closure owns `name`, which implements `Clone`, so it can be called repeatedly.

Common use case: passing closures to threads or async tasks that must outlive their parent scope.

What category of types do closures belong to?

6.4. Internal representation of closures

What category of types do closures belong to?

anonymous types. Each closure has a unique, unnamed type generated by the compiler.

Closures are like structs with a function field:

1. Each captured variable is a reference field.
2. The closure body is a method that uses these fields.

For example, the closure `|x| x + offset` where `offset` is captured from the environment is represented as:

```
1 struct Closure {  
2     offset: &i32,  
3 }  
4  
5 impl Closure {  
6     fn call(&self, x: i32) -> i32 {  
7         x + *self.offset  
8     }  
9 }
```

([playground link](#))

6.4. Internal representation of closures

What category of types do closures belong to?

anonymous types. Each closure has a unique, unnamed type generated by the compiler.

Closures are like structs with a function field:

1. Each captured variable is a reference field.
2. The closure body is a method that uses these fields.

For example, the closure `|x| x + offset` where `offset` is captured from the environment is represented as:

```
1 struct Closure {  
2     offset: &i32,  
3 }  
4  
5 impl Closure {  
6     fn call(&self, x: i32) -> i32 {  
7         x + *self.offset  
8     }  
9 }
```

[\(playground link\)](#)

Warning

The thing that sets closures apart is the hidden signature of the `call` method (whether it takes `&self` or not).

6.5. Datastructures containing functions

To store a closure in a data structure, we need to turn the anonymous closure type into generic type variable.

What does `Fn(i32) -> i32` mean here?

```
1 struct Processor<F> {  
2     field: i32  
3     func: F,  
4 }  
5  
6 impl<F> Processor<F>  
7 where  
8     F: Fn(i32) -> i32,  
9 {  
10     fn process(&self) -> i32 {  
11         (self.func)(self.field)  
12     }  
13 }
```

(playground link)

6.5. Datastructures containing functions

To store a closure in a data structure, we need to turn the anonymous closure type into generic type variable.

```
1 struct Processor<F> {  
2     field: i32  
3     func: F,  
4 }  
5  
6 impl<F> Processor<F>  
7 where  
8     F: Fn(i32) -> i32,  
9 {  
10     fn process(&self) -> i32 {  
11         (self.func)(self.field)  
12     }  
13 }
```

(playground link)

What does `Fn(i32) -> i32` mean here?

It's an **Fn-trait bound** (function trait bound) indicating that `F` must be a type that implements the `Fn` trait.

Always use generic `Fn` trait bounds for structs containing functions.

6.5. Datastructures containing functions

To store a closure in a data structure, we need to turn the anonymous closure type into generic type variable.

```
1 struct Processor<F> {  
2     field: i32  
3     func: F,  
4 }  
5  
6 impl<F> Processor<F>  
7 where  
8     F: Fn(i32) -> i32,  
9 {  
10     fn process(&self) -> i32 {  
11         (self.func)(self.field)  
12     }  
13 }
```

(playground link)

What does `Fn(i32) -> i32` mean here?

It's an **Fn-trait bound** (function trait bound) indicating that `F` must be a type that implements the `Fn` trait.

Always use generic `Fn` trait bounds for structs containing functions.

Warning

Do not turn your closure into a **function pointer** (fn type), even when the compiler suggests it!

Why are function pointers bad?

6.5. Datastructures containing functions

To store a closure in a data structure, we need to turn the anonymous closure type into generic type variable.

```
1 struct Processor<F> {  
2     field: i32  
3     func: F,  
4 }  
5  
6 impl<F> Processor<F>  
7 where  
8     F: Fn(i32) -> i32,  
9 {  
10     fn process(&self) -> i32 {  
11         (self.func)(self.field)  
12     }  
13 }
```

(playground link)

What does `Fn(i32) -> i32` mean here?

It's an **Fn-trait bound** (function trait bound) indicating that `F` must be a type that implements the `Fn` trait.

Always use generic `Fn` trait bounds for structs containing functions.

Warning

Do not turn your closure into a **function pointer** (`fn` type), even when the compiler suggests it!

Why are function pointers bad?

May need heap allocation. Cannot capture variables.

6.6. Practice: Rustlings Iterators

Rustlings exercises:

- `exercises/18_iterators/iterators1.rs` - Basic iterator usage
- `exercises/18_iterators/iterators2.rs` - Iterator methods
- `exercises/18_iterators/iterators3.rs` - `map` and `collect`
- `exercises/18_iterators/iterators4.rs` - Factorial with `fold`
- `exercises/18_iterators/iterators5.rs` - Complex iterator chains

6.7. Practice: Exercism Accumulate

Implement your own map function using closures:

Exercism: Accumulate:

- Path: `exercises/practice/accumulate/`
- Goal: Implement a generic map function
- Requires: Closures, generics, function trait bounds

Info

Great for understanding how closures work as function parameters!

Online: <https://exercism.org/tracks/rust/exercises/accumulate>



6.8. Closure traits

Please open demo file `session-2/examples/s2e5-closure-traits.rs` and play around with the code.

6.8. Closure traits

Closures have anonymous types that implement one or more of these traits:

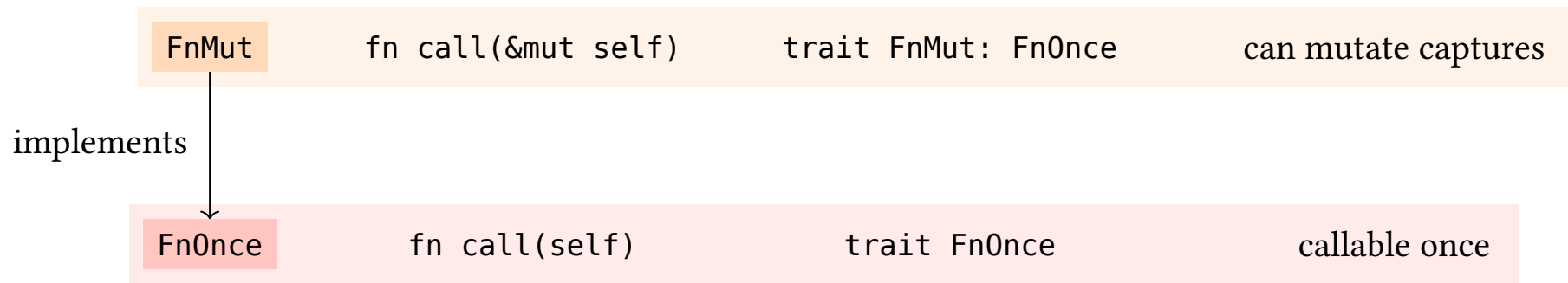
6.8. Closure traits

Closures have anonymous types that implement one or more of these traits:

`FnOnce``fn call(self)``trait FnOnce``callable once`

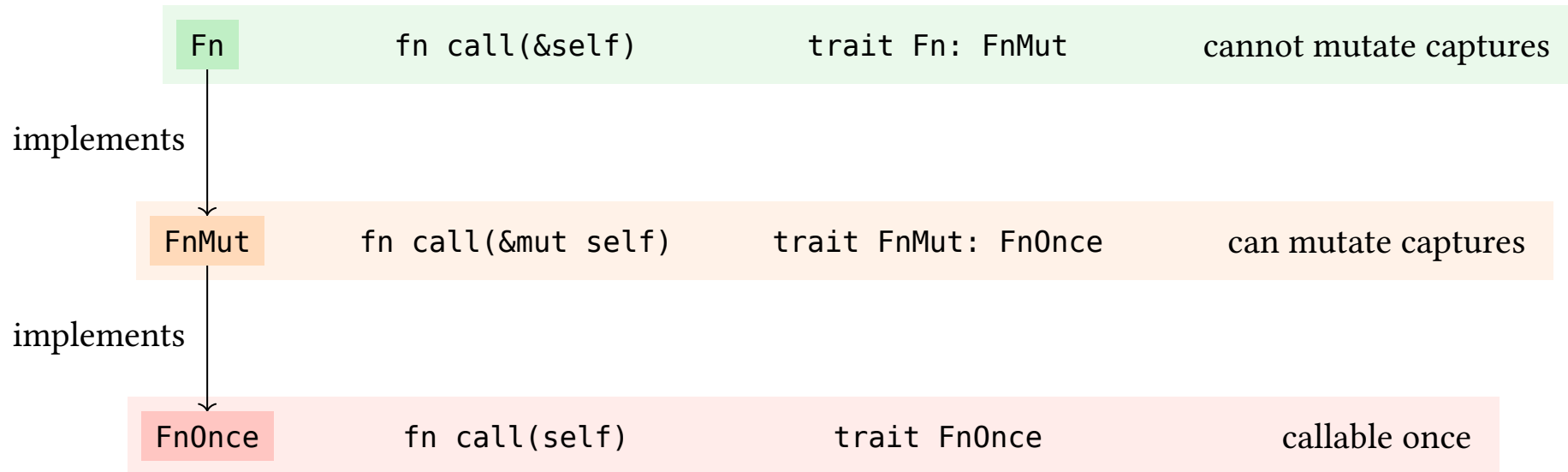
6.8. Closure traits

Closures have anonymous types that implement one or more of these traits:



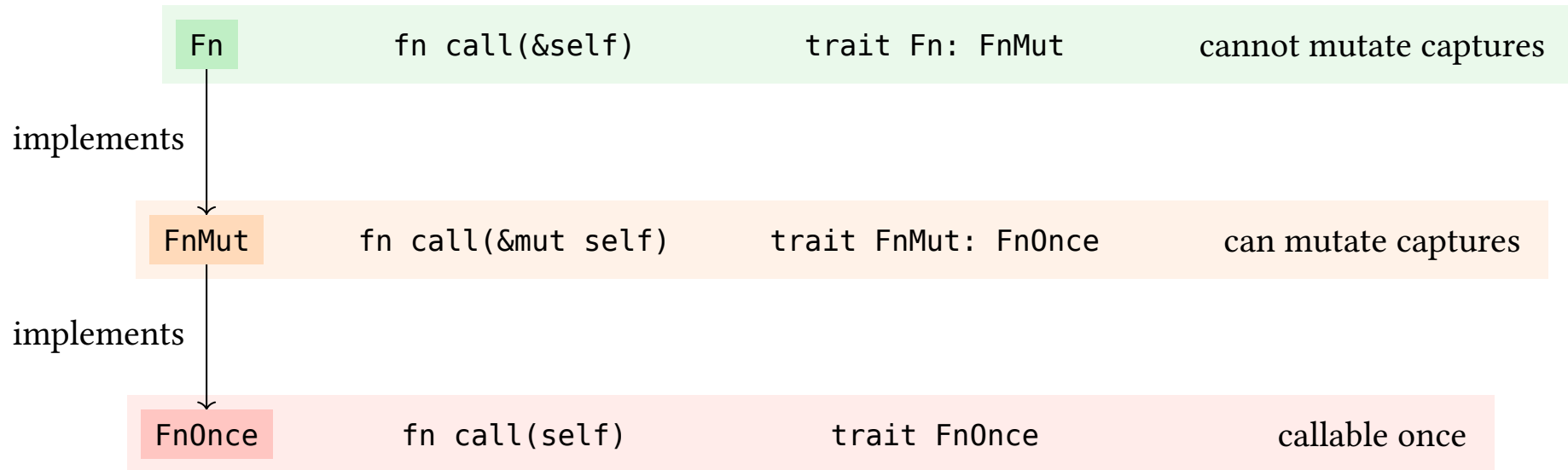
6.8. Closure traits

Closures have anonymous types that implement one or more of these traits:



6.8. Closure traits

Closures have anonymous types that implement one or more of these traits:

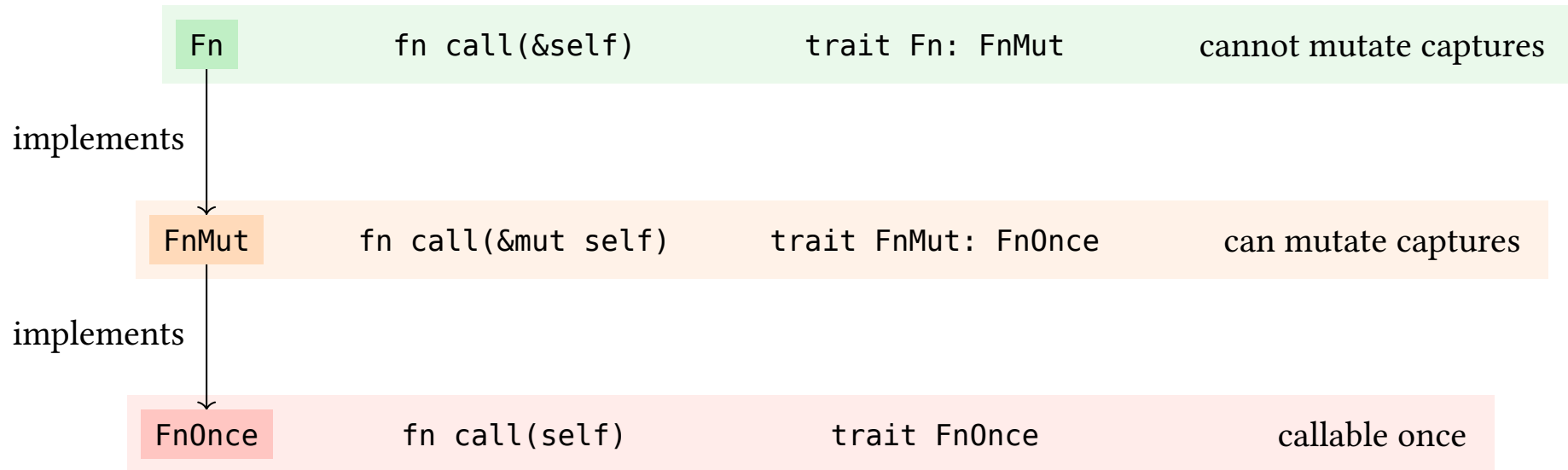


Info

Trait hierarchy: `Fn` implements `FnMut`, `FnMut` implements `FnOnce`. A closure callable with `&self` can also be called with `&mut self` or `self`.

6.8. Closure traits

Closures have anonymous types that implement one or more of these traits:



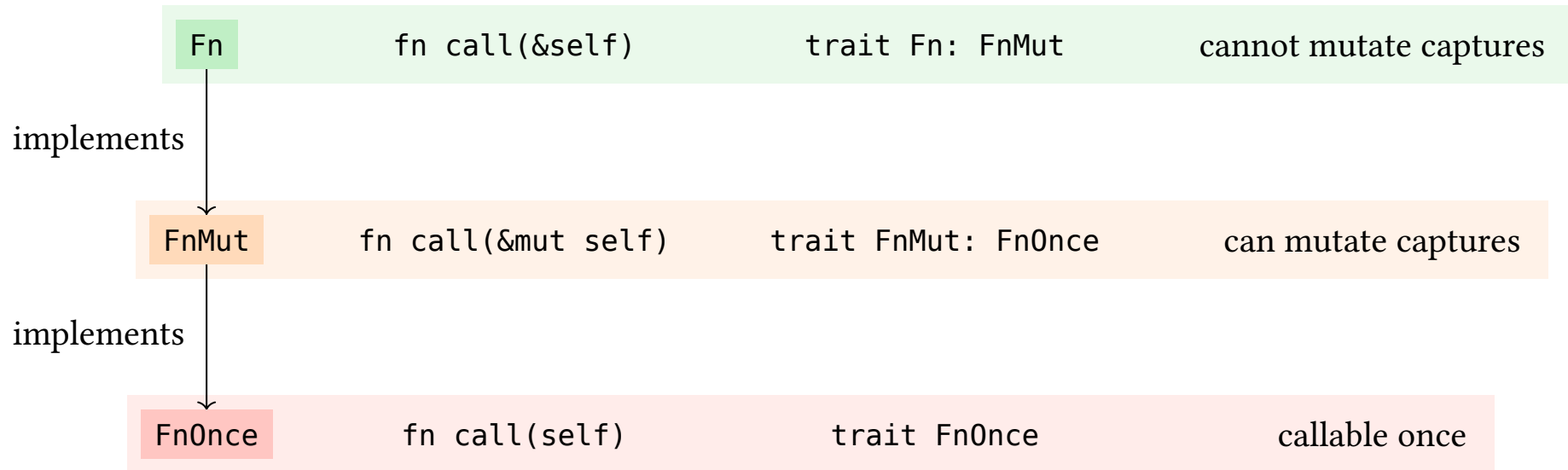
Info

Trait hierarchy: `Fn` implements `FnMut`, `FnMut` implements `FnOnce`. A closure callable with `&self` can also be called with `&mut self` or `self`.

Can a closure implementing `Fn` be used where `FnOnce` is required?

6.8. Closure traits

Closures have anonymous types that implement one or more of these traits:



Info

Trait hierarchy: `Fn` implements `FnMut`, `FnMut` implements `FnOnce`. A closure callable with `&self` can also be called with `&mut self` or `self`.

Can a closure implementing `Fn` be used where `FnOnce` is required?

Yes. `Fn` is a subtrait of `FnMut`, which is a subtrait of `FnOnce`.



6.9. Picking the right closure trait

You define a function accepting a closure (a higher-order function)

Which closure trait should use as a generic type trait bound?

6.9. Picking the right closure trait

You define a function accepting a closure (a higher-order function)

Which closure trait should use as a generic type trait bound?

Prefer `FnOnce` (accepts any closure type).

Why?

6.9. Picking the right closure trait

You define a function accepting a closure (a higher-order function)

Which closure trait should use as a generic type trait bound?

Prefer `FnOnce` (accepts any closure type).

Why?

`Fn` and `FnMut` both implement `FnOnce`, so all closures can be passed. Most flexible for callers.

6.9. Picking the right closure trait

You define a function accepting a closure (a higher-order function)

Which closure trait should use as a generic type trait bound?

Prefer `FnOnce` (accepts any closure type).

Why?

`Fn` and `FnMut` both implement `FnOnce`, so all closures can be passed. Most flexible for callers.

```
1 fn apply_once<F: FnOnce(i32) -> i32>(f: F, value: i32) -> i32 {  
2     f(value)  
3 }  
4  
5 fn main() {  
6     apply_once(|n| n + 1, 5);           // Fn works  
7     apply_once(move |n| n * 2, 5);     // FnOnce works  
8 }
```

(playground link)

6.9. Picking the right closure trait

You have to provide a closure for someone else's higher-order function

Which closure trait should your closure implement?

6.9. Picking the right closure trait

You have to provide a closure for someone else's higher-order function

Which closure trait should your closure implement?

Aim for `Fn` if possible, fall back to `FnMut` or `FnOnce` if needed.

Why?

6.9. Picking the right closure trait

You have to provide a closure for someone else's higher-order function

Which closure trait should your closure implement?

Aim for Fn if possible, fall back to FnMut or FnOnce if needed.

Why?

Fn closures work wherever FnMut or FnOnce is required, giving maximum usability.
Think of receiver types: `&self (Fn)` works where `&mut self (FnMut)` or `self (FnOnce)` is needed.

6.9. Picking the right closure trait

You have to provide a closure for someone else's higher-order function

Which closure trait should your closure implement?

Aim for Fn if possible, fall back to FnMut or FnOnce if needed.

Why?

Fn closures work wherever FnMut or FnOnce is required, giving maximum usability.
Think of receiver types: &self (Fn) works where &mut self (FnMut) or self (FnOnce) is needed.

```
1 fn process<F: Fn(i32) -> i32>(items: &[i32], f: F) {
2     for item in items {
3         println!("{}", f(*item)); // Requires Fn: called multiple times
4     }
5 }
6
7 fn main() {
8     process(&[1, 2, 3], |x| x * 2); // Fn: OK (no mutation)
9     // let mut c = 0;
10    // process(&[1, 2, 3], |x| { c += 1; x }); // FnMut: Error!
11 }
```

(playground link)



What are auto-traits?

6.10. Auto-traits of closures

What are auto-traits?

Traits that the compiler can automatically implement for types based on their structure.

Closures automatically implement `Copy` and `Clone` based on captured values:

6.10. Auto-traits of closures

What are auto-traits?

Traits that the compiler can automatically implement for types based on their structure.

Closures automatically implement `Copy` and `Clone` based on captured values:

```
let x = String::from("data");  
      captures  
let f = move || { x.clone() };
```

6.10. Auto-traits of closures

What are auto-traits?

Traits that the compiler can automatically implement for types based on their structure.

Closures automatically implement `Copy` and `Clone` based on captured values:

```
let x = String::from("data");
```

captures

```
let f = move || { x.clone() };
```

auto-trait

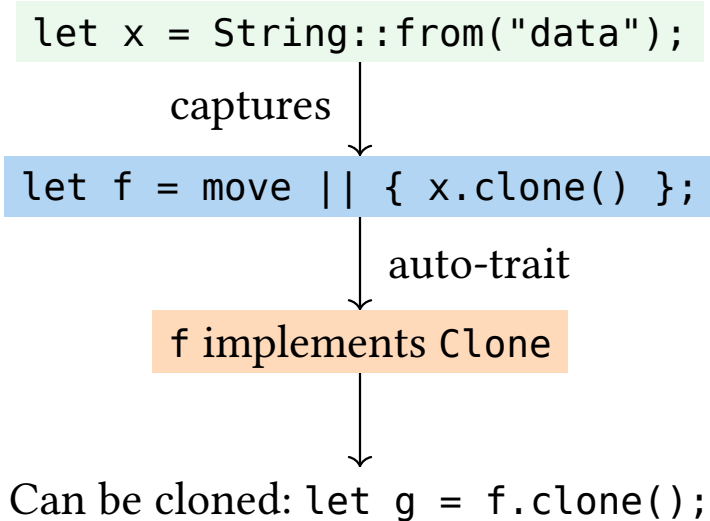
f implements Clone

6.10. Auto-traits of closures

What are auto-traits?

Traits that the compiler can automatically implement for types based on their structure.

Closures automatically implement `Copy` and `Clone` based on captured values:



6.10. Auto-traits of closures

What are auto-traits?

Traits that the compiler can automatically implement for types based on their structure.

Closures automatically implement `Copy` and `Clone` based on captured values:

```
let x = String::from("data");
```

captures

```
let f = move || { x.clone() };
```

auto-trait

f implements `Clone`

Can be cloned: `let g = f.clone();`

Info

Closures inherit properties from the values they capture.

6.11. Exercise

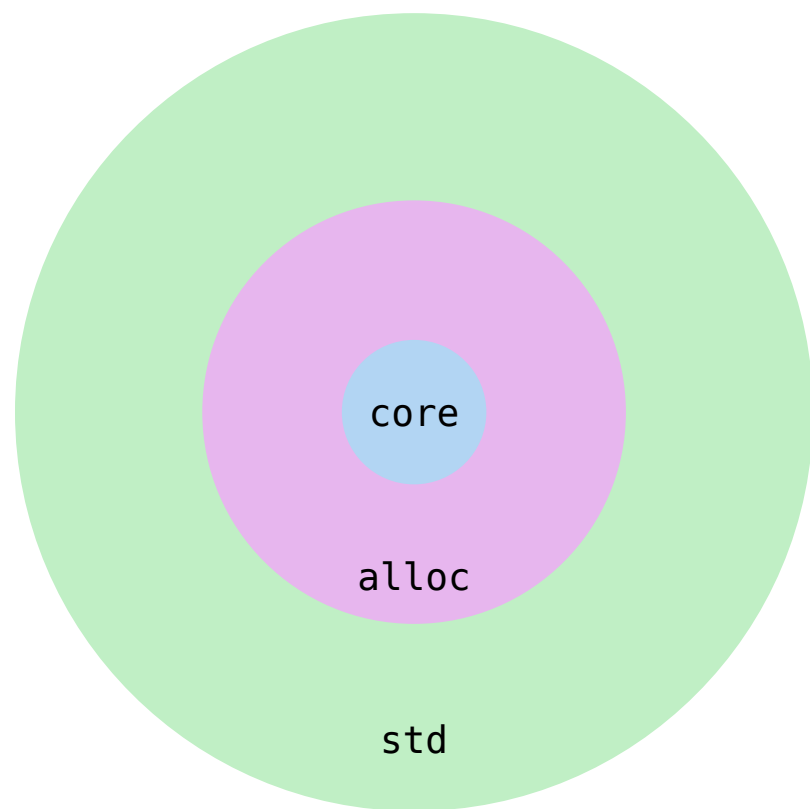
Warning**Important Exercise**

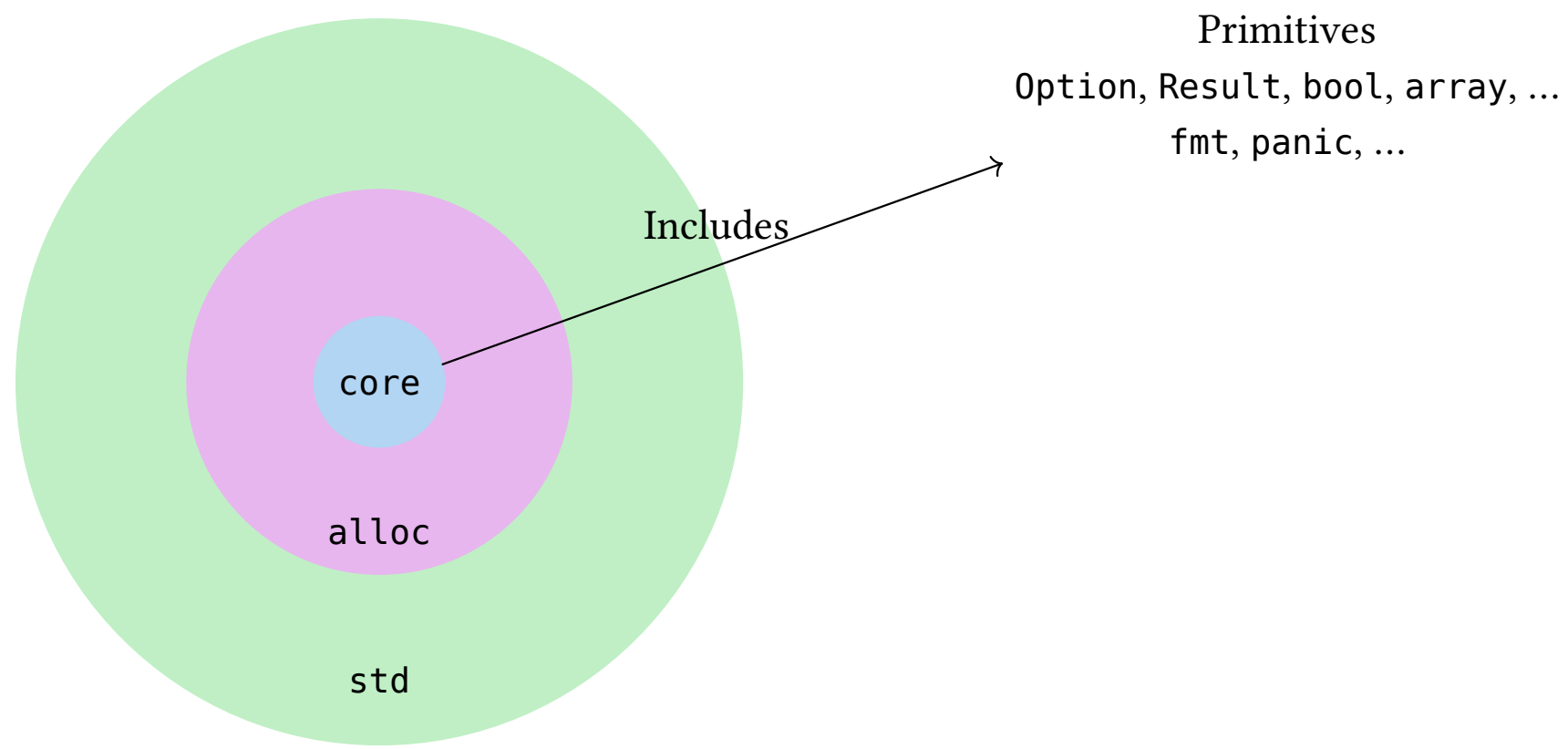
Please find the statement of the exercise at `session-2/examples/s2d6-closures.rs` in this repo.

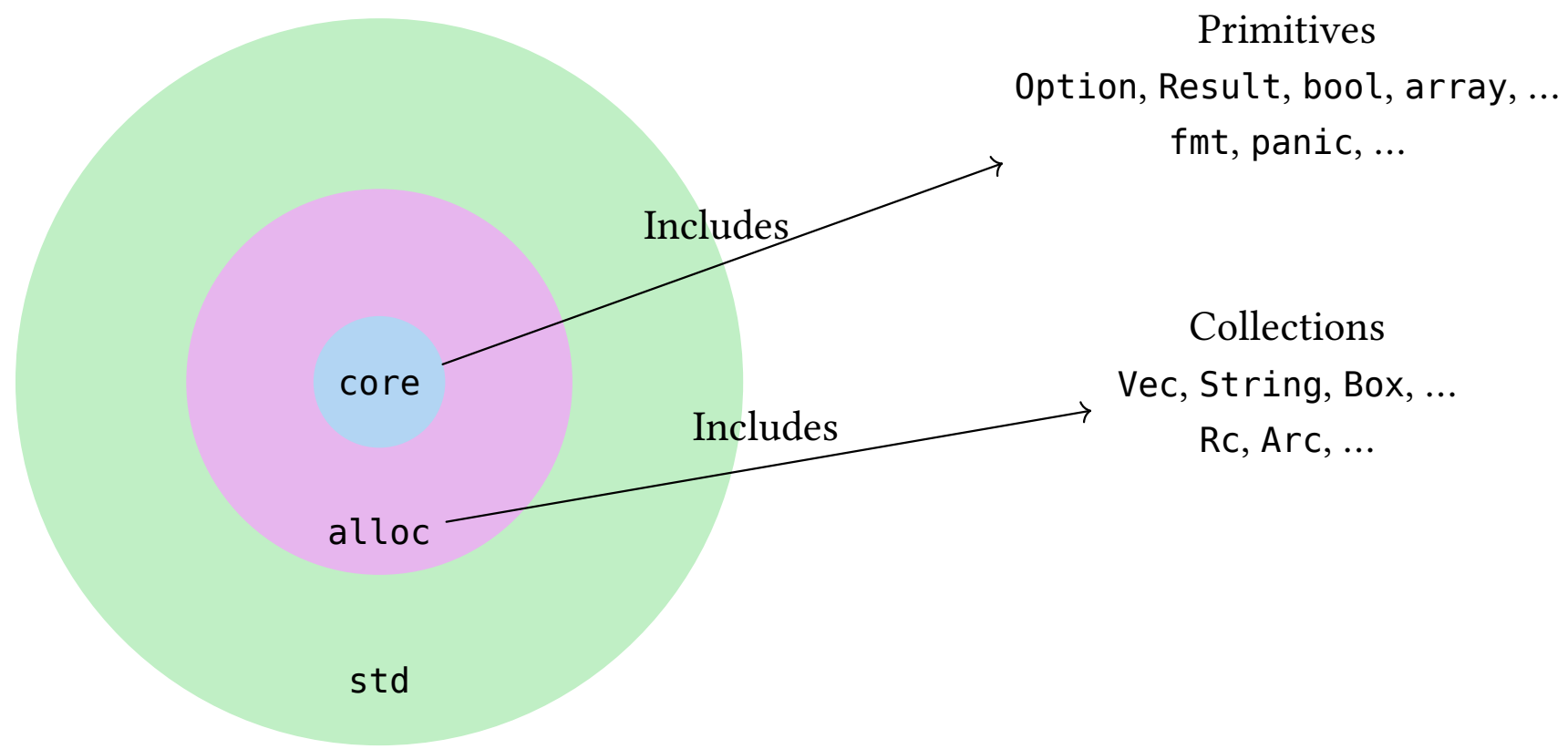
Run it with `cargo run --example s2d6-closures`.

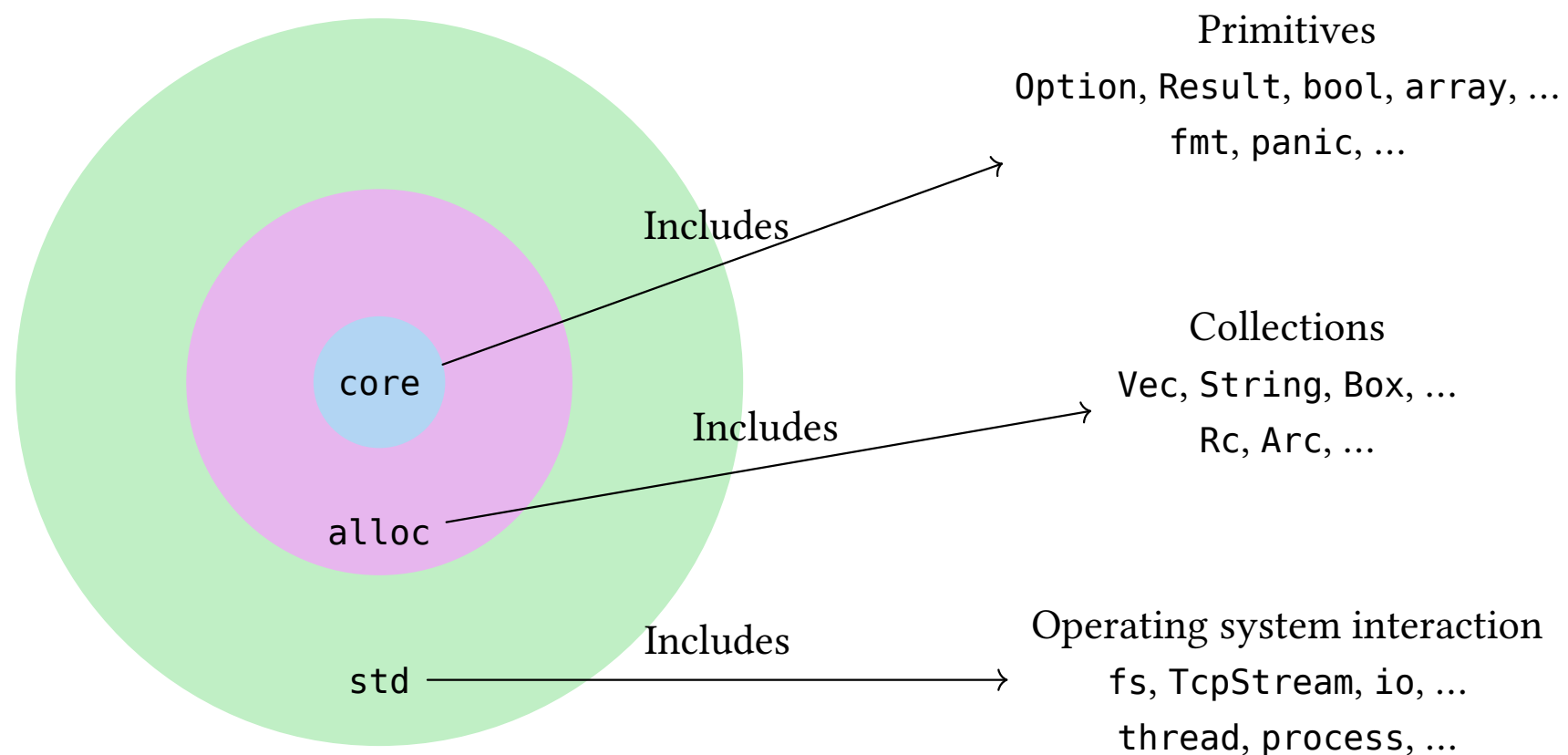
1. Quiz about last lecture	1
2. Pattern matching	8
3. Tooling intermezzo	25
4. Methods and traits	29
5. Generics	48
6. Closures	69
7. Standard library types	85
7.1. Overview	87
7.2. Documentation resources	89
7.3. Creating documentation	90
7.4. The Option type	94
7.5. Result type	101
7.6. Heap allocation with alloc	103
7.7. String	104
7.8. Practice: Exercism Luhn Trait	106
7.9. Vec type	108
7.10. Practice: Exercism Nucleotide Count	109
7.11. Exercise	110

8. Conclusion	111
---------------------	-----









7.1. Overview

It is good style to prefer the minimal amount of imports:

- If it is available in `core`, use that
- Else, if it is available in `alloc`, use that
- Else, use `std`

7.1. Overview

It is good style to prefer the minimal amount of imports:

- If it is available in `core`, use that
- Else, if it is available in `alloc`, use that
- Else, use `std`

What would be a practical way to enforce this?

7.1. Overview

It is good style to prefer the minimal amount of imports:

- If it is available in `core`, use that
- Else, if it is available in `alloc`, use that
- Else, use `std`

What would be a practical way to enforce this?

Forbid certain lints in your `Cargo.toml` file.

```
1 [lints.clippy]
2 alloc_instead_of_core = "deny"
3 std_instead_of_alloc = "deny"
4 std_instead_of_core = "deny"
```

(playground link)

7.2. Documentation resources

Locally:

- `rustup doc --std` (opens standard library docs)
- `cargo doc --open`
- `cargo doc --open --no-deps` (only your code)

Try these commands out!

7.2. Documentation resources

Locally:

- `rustup doc --std` (opens standard library docs)
- `cargo doc --open`
- `cargo doc --open --no-deps` (only your code)

Try these commands out!

Online:

- <https://doc.rust-lang.org/stable/std/>
- <https://std.rs>

7.3. Creating documentation

7.3.1. Funtion documentation

You can add “doc-comments” to your code using triple slashes ///:

```
1  /// Determine whether the first argument is divisible by the second argument.
2  ///
3  /// If the second argument is zero, the result is false.
4  fn is_divisible_by(lhs: u32, rhs: u32) -> bool {
5      if rhs == 0 {
6          return false;
7      }
8      lhs % rhs == 0
9  }
```

(playground link)

Will be shown when you run `cargo doc --open` (or hover over the function in your editor).

7.3. Creating documentation

7.3.2. More doc-comment positions

Enum variants and struct fields can be documented too:

```
1  /// A simple enum.  
2  enum MyEnum {  
3      /// The first variant.  
4      VariantA,  
5      /// The second variant.  
6      VariantB,  
7  }
```

(playground link)

7.3. Creating documentation

7.3.3. Documenting modules

To document the module containing the current definition, use `//!` comments at the top of the file:

```
1  //! This module contains utility functions for mathematical operations.
2  pub fn add(a: i32, b: i32) -> i32 {
3          a + b
4  }
```

[\(playground link\)](#)

All text in `///` or `//!` comments supports Markdown.

You can even **write unit tests in doc-comments** that are checked when you run `cargo test`!

```
1  /// Adds two numbers together.
2  ///
3  /// # Examples
4  /// ```
5  /// let sum = add(2, 3);
6  /// assert_eq!(sum, 5);
7  /// ```
```

[\(playground link\)](#)

7.3. Creating documentation

7.3.4. Demonstration

Every library that you publish receive automatic documentation on `docs.rs`.

See for example:

- https://docs.rs/clone-stream/0.4.0/clone_stream/
- <https://docs.rs/rand/latest/rand/>

Every public crate has a link to the documentation.

7.4. The Option type

Option is part of core and represents an optional value:

```
1 fn main() {  
2     let name = "Löwe 老虎 Léopard Gepardi";  
3     let mut position: Option<usize> = name.find('é');  
4     dbg!(position);  
5     assert_eq!(position.unwrap(), 14);  
6     position = name.find('Z');  
7     dbg!(position);  
8     assert_eq!(position.expect("Character not found"), 0);  
9 }
```

(playground link)

String::find returns Option. Unwrapping an Option that is None **will cause a panic**.

Is it possible to forget handling a None case?

7.4. The Option type

Option is part of core and represents an optional value:

```
1 fn main() {  
2     let name = "Löwe 老虎 Léopard Gepardi";  
3     let mut position: Option<usize> = name.find('é');  
4     dbg!(position);  
5     assert_eq!(position.unwrap(), 14);  
6     position = name.find('Z');  
7     dbg!(position);  
8     assert_eq!(position.expect("Character not found"), 0);  
9 }
```

(playground link)

String::find returns Option. Unwrapping an Option that is None **will cause a panic**.

Is it possible to forget handling a None case?

No, the compiler forces you to handle both cases.

7.4. The Option type

7.4.1. Use cases for Option

Option is really about data or information that might be absent.

Warning

The `Option::None` variant should **not replace every null** value in other languages!

In most cases, you should declare a custom enum:

```
1 enum LoginResult {  
2     Success(UserData),  
3     InvalidPassword,  
4     UserNotFound,  
5 }
```

(playground link)

Why would it be better to use a custom enum instead of `Option<UserData>`?

7.4. The Option type

7.4.1. Use cases for Option

Option is really about data or information that might be absent.

Warning

The `Option::None` variant should **not replace every `null`** value in other languages!

In most cases, you should declare a custom enum:

```
1 enum LoginResult {  
2     Success(UserData),  
3     InvalidPassword,  
4     UserNotFound,  
5 }
```

(playground link)

Why would it be better to use a custom enum instead of `Option<UserData>`?

A custom enum can provide more context about why the data is absent

7.4. The `Option` type

7.4.2. Relationship `bool` and `Option`

The standard library API entangles `bool` and `Option` in interesting ways.

7.4. The Option type

7.4.2. Relationship `bool` and `Option`

The standard library API entangles `bool` and `Option` in interesting ways.

`bool::then()` converts a boolean to an `Option`:

```
1 let is_valid = true;
2 assert_eq!(is_valid.then(|| 42), Some(42));
3
4 let is_invalid = false;
5 assert_eq!(is_invalid.then(|| 42), None);
```

([playground link](#))

7.4. The Option type

7.4.2. Relationship `bool` and `Option`

The standard library API entangles `bool` and `Option` in interesting ways.

`bool::then()` converts a boolean to an `Option`:

```
1 let is_valid = true;
2 assert_eq!(is_valid.then(|| 42), Some(42));
3
4 let is_invalid = false;
5 assert_eq!(is_invalid.then(|| 42), None);
```

[\(playground link\)](#)

`and()` combines `Options` like boolean AND (`&&`):

```
1 let x = Some(2);
2 let y: Option<i32> = None;
3 assert_eq!(x.and(y), None); // Returns None if either is None
```

[\(playground link\)](#)

7.4. The Option type

7.4.3. The actual machine representation

We'll use transmute to peek at the internal bit representation:

```
1 use std::mem::transmute;
2 macro_rules! dbg_bits {
3     ($e:expr, $bit_type:ty) => {
4         println!("- {}: {:#x}", stringify!($e), transmute::<_, $bit_type>($e));
5     };
6 }
```

(playground link)

For every expression, what is its bit representation (as a hexadecimal number)?

7.4. The Option type

7.4.3. The actual machine representation

We'll use transmute to peek at the internal bit representation:

```
1 use std::mem::transmute;  
2 macro_rules! dbg_bits {  
3     ($e:expr, $bit_type:ty) => {  
4         println!("- {}: {:#x}", stringify!($e), transmute::<_, $bit_type>($e));  
5     };  
6 }
```

[\(playground link\)](#)

For every expression, what is its bit representation (as a hexadecimal number)?

Warning

transmute is unsafe and reinterprets memory.

7.4. The Option type

```
1  unsafe {  
2      println!("bool:");  
3      dbg_bits!(false, u8);  
4      dbg_bits!(true, u8);  
5  
6      println!("Option<bool>:");  
7      dbg_bits!(None::<bool>, u8);  
8      dbg_bits!(Some(false), u8);  
9      dbg_bits!(Some(true), u8);  
10 }
```

(playground link)

Expected output:

```
1  bool:  
2  - false: 0x0  
3  - true: 0x1  
4  Option<bool>:  
5  - None::<bool>: 0x2  
6  - Some(false): 0x0  
7  - Some(true): 0x1
```

(playground link)

Info

Option<bool> uses 0x2 for None, allowing all three states in 1 byte!

7.4. The Option type

```

1  unsafe {
2      println!("Option<Option<bool>>:");
3      dbg_bits!(Some(Some(false)), u8);
4      dbg_bits!(Some(Some(true)), u8);
5      dbg_bits!(Some(None::<bool>), u8);
6      dbg_bits!(None::<Option<bool>>,
7                u8);
8
9
10                                     (playground link)

```

Expected output:

```

1  Option<Option<bool>>:
2  - Some(Some(false)): 0x0
3  - Some(Some(true)): 0x1
4  - Some(None::<bool>): 0x2
5  - None::<Option<bool>>: 0x3 (playground link)

```

Info

Four distinct states, still fitting in just 1 byte!

7.4. The `Option` type

What is the “niche optimisation” in the context of `Option`?

7.4. The Option type

What is the “niche optimisation” in the context of Option?

Certain types have unused bit patterns (niches) that can be used to represent None without extra space.

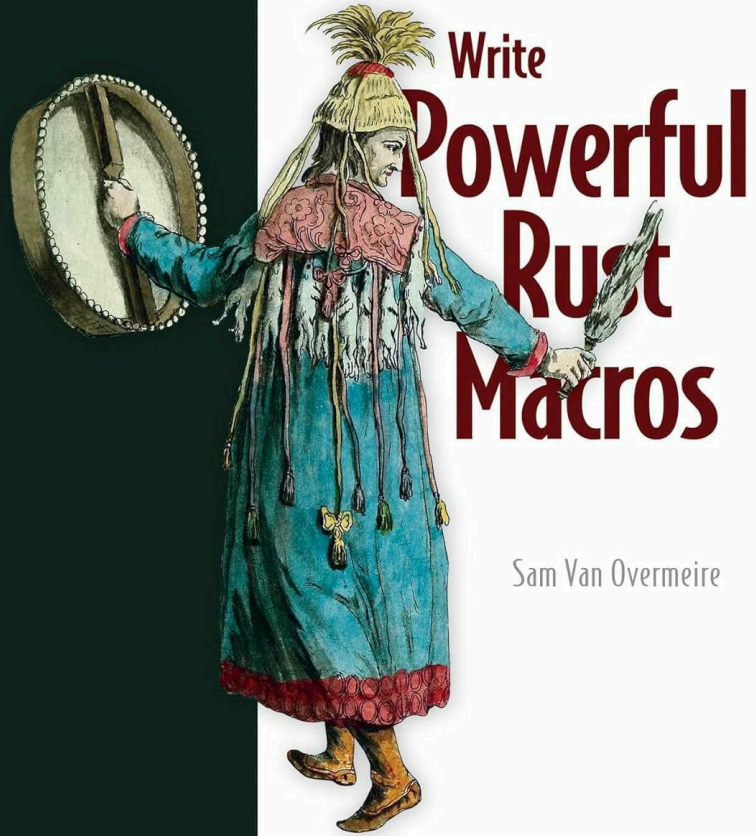
```
1 unsafe {
2     println!("Option<&i32>:");
3     dbg_bits!(None:::<&i32>, usize);
4     dbg_bits!(Some(&0i32), usize);
5 }
6
7
8 (playground link)
```

Expected output (on 64-bit):

```
1 Option<&i32>:
2 - None:::<&i32>: 0x0
3 - Some(&0i32): 0x7ffc8e4a1b2c (playground link)
```

Info

None is represented as null pointer (0x0), while Some contains the actual memory address. No extra space needed for the discriminant!



Sam Van Overmeire

 MANNING

7.5. Result type

```
1  use std::fs::File;
2  use std::io::Read;
3
4  fn main() {
5      let file: Result<File, std::io::Error> = File::open("diary.txt");
6      match file {
7          Ok(mut file) => {
8              let mut contents = String::new();
9              if let Ok(bytes) = file.read_to_string(&mut contents) {
10                  println!("Dear diary: {contents} ({bytes} bytes)");
11              } else {
12                  println!("Could not read file content");
13              }
14          }
15          Err(err) => {
16              println!("The diary could not be opened: {err}");
17          }
18      }
19 }
```

[\(playground link\)](#)

7.5. Result type

`Result` is similar to `Option`, but indicates the success or failure of an operation.

7.5. Result type

`Result` is similar to `Option`, but indicates the success or failure of an operation.

`Result` is generic: `Result<T, E>` where:

- `T` is used in the `Ok` variant and
- `E` appears in the `Err` variant.

Rule of thumb

Use `Result` instead of `Option` when the reason of failure needs different handling.

What do `Result` and `Option` have in common?

7.5. Result type

`Result` is similar to `Option`, but indicates the success or failure of an operation.

`Result` is generic: `Result<T, E>` where:

- `T` is used in the `Ok` variant and
- `E` appears in the `Err` variant.

Rule of thumb

Use `Result` instead of `Option` when the reason of failure needs different handling.

What do `Result` and `Option` have in common?

They **may short-circuit** the execution of a function.

What is the name for a object that has short-circuiting behaviour?

7.5. Result type

Result is similar to Option, but indicates the success or failure of an operation.

Result is generic: `Result<T, E>` where:

- T is used in the Ok variant and
- E appears in the Err variant.

Rule of thumb

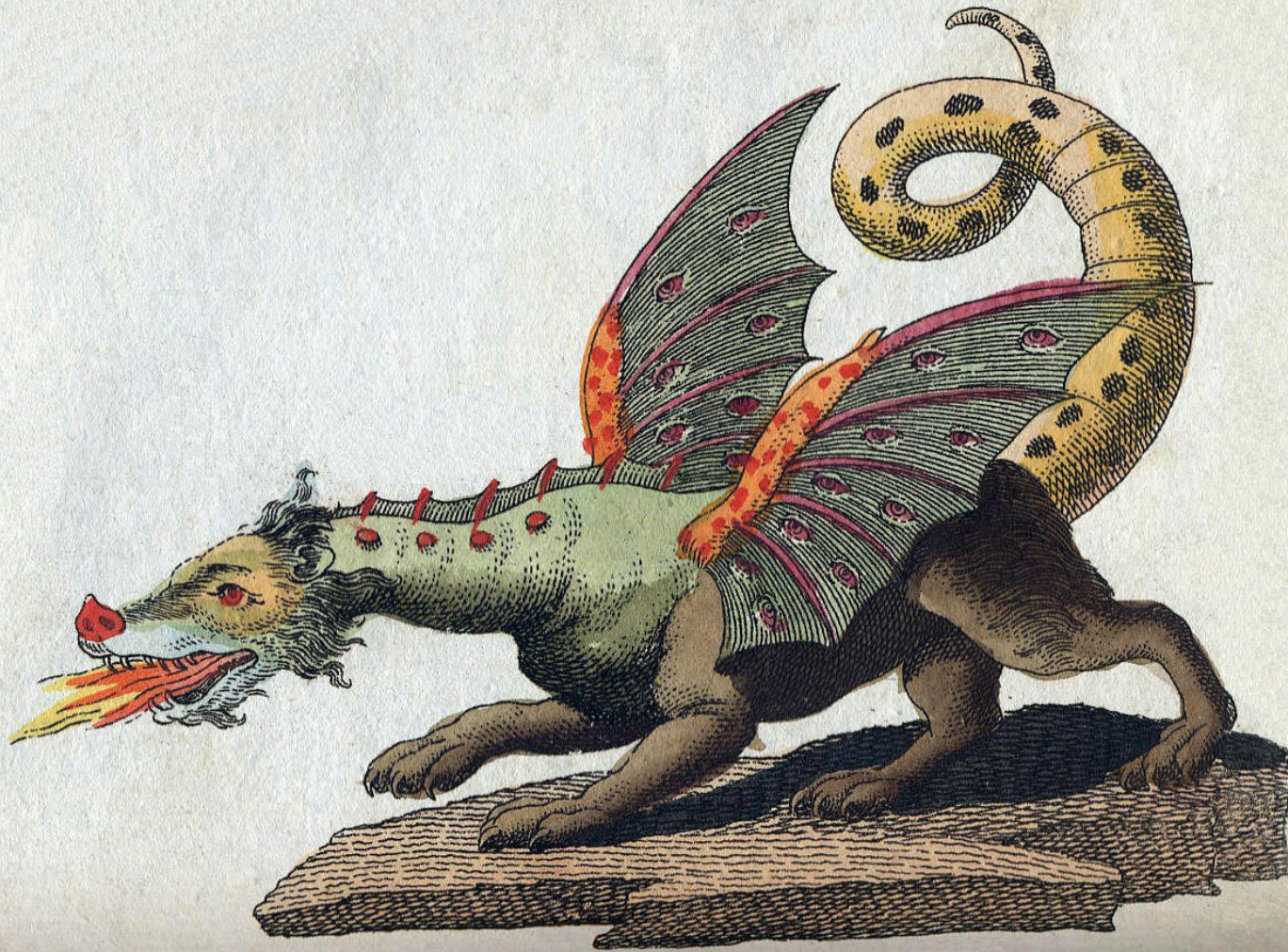
Use Result instead of Option when the reason of failure needs different handling.

What do Result and Option have in common?

They **may short-circuit** the execution of a function.

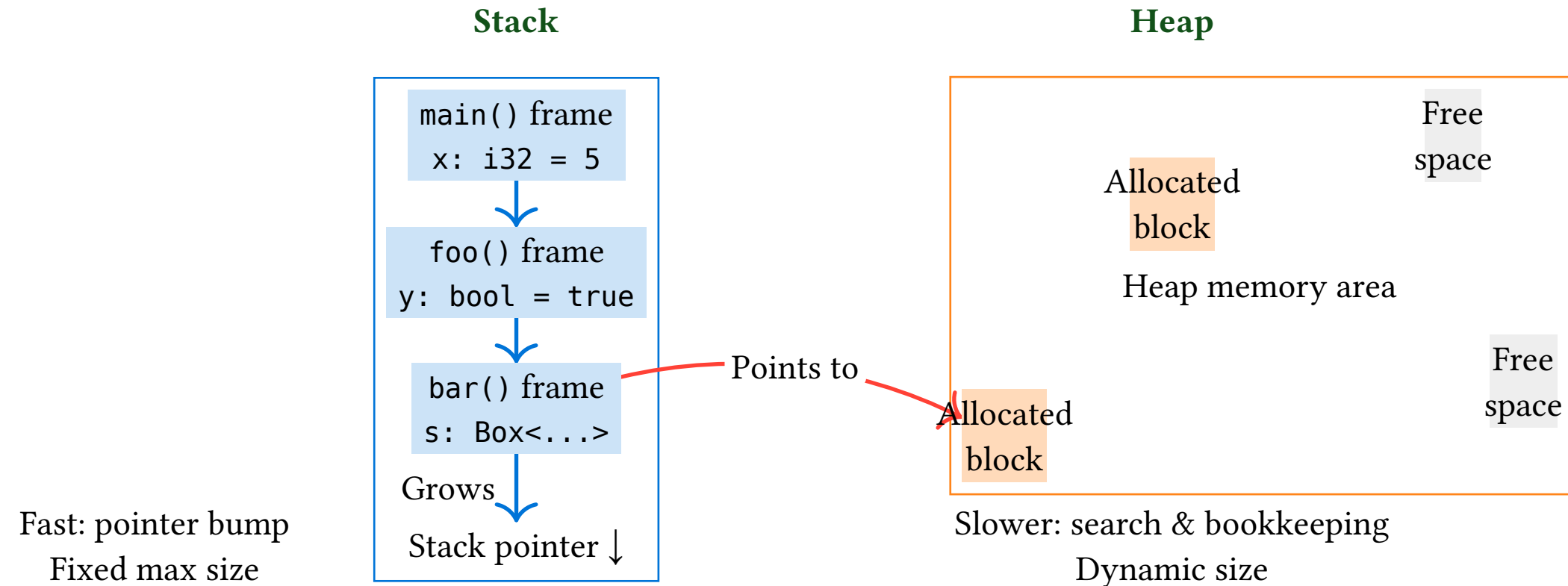
What is the name for a object that has short-circuiting behaviour?

A Monad.



Warning

We have not yet done any heap allocation!



7.7. String

String is a growable UTF-8 encoded string:

```
1  fn main() {
2      let mut s1 = String::new();
3      s1.push_str("Hello");
4      println!("s1: len = {}, capacity = {}", s1.len(), s1.capacity());
5
6      let mut s2 = String::with_capacity(s1.len() + 1);
7      s2.push_str(&s1);
8      s2.push('!');
9      println!("s2: len = {}, capacity = {}", s2.len(), s2.capacity());
10
11     let s3 = String::from("🇳🇱 ");
12     println!("s3: len = {}, number of chars = {}", s3.len(), s3.chars().count());
13 }
```

(playground link)

How is `String` stored internally?

How is `String` stored internally?

Just a safe wrapper around a vector of bytes on the heap.

- `String::new` returns a new empty string, use `String::with_capacity` when you know how much data you want to push to the string.

7.7. String

How is `String` stored internally?

Just a safe wrapper around a vector of bytes on the heap.

- `String::new` returns a new empty string, use `String::with_capacity` when you know how much data you want to push to the string.
- `String::len` returns byte length (which can be different from its length in characters).

7.7. String

How is String stored internally?

Just a safe wrapper around a vector of bytes on the heap.

- `String::new` returns a new empty string, use `String::with_capacity` when you know how much data you want to push to the string.
- `String::len` returns byte length (which can be different from its length in characters).
- `String::chars` returns an iterator over the actual characters.

Warning

Note that a char can be different from what a human will consider a “character” due to grapheme clusters.

Most String methods are actually defined on the thing String “dereferences” to: `&str`!

Do you remember what `&str` is?

7.7. String

How is String stored internally?

Just a safe wrapper around a vector of bytes on the heap.

- `String::new` returns a new empty string, use `String::with_capacity` when you know how much data you want to push to the string.
- `String::len` returns byte length (which can be different from its length in characters).
- `String::chars` returns an iterator over the actual characters.

Warning

Note that a char can be different from what a human will consider a “character” due to grapheme clusters.

Most String methods are actually defined on the thing String “dereferences” to: `&str`!

Do you remember what `&str` is?

A string slice, a view into a UTF-8 encoded string stored elsewhere.

7.8. Practice: Exercism Luhn Trait

Exercism: Luhn Trait:

- Path: `exercises/practice/luhn-trait/`
- Goal: Implement Luhn algorithm using a custom trait
- Requires: Generic trait implementation for multiple types

Online: <https://exercism.org/tracks/rust/exercises/luhn-trait>

7.8. Practice: Exercism Luhn Trait

7.8.1. Creating strings

In practice, `String` instances are generated with the `to_string` or `into` method from compile-time static string slices:

```
1 let s1: String = "Hello, world!".to_string();
```

(playground link)

7.8. Practice: Exercism Luhn Trait

7.8.1. Creating strings

In practice, `String` instances are generated with the `to_string` or `into` method from compile-time static string slices:

```
1 let s1: String = "Hello, world!".to_string();
```

[\(playground link\)](#)

7.8.2. String interpolation

Always done with macros:

```
1 println!("s3: len = {}, number of chars = {}", s3.len(), s3.chars().count());
```

[\(playground link\)](#)

7.8. Practice: Exercism Luhn Trait

7.8.1. Creating strings

In practice, `String` instances are generated with the `to_string` or `into` method from compile-time static string slices:

```
1 let s1: String = "Hello, world!".to_string();
```

[\(playground link\)](#)

7.8.2. String interpolation

Always done with macros:

```
1 println!("s3: len = {}, number of chars = {}", s3.len(), s3.chars().count());
```

[\(playground link\)](#)

Try to use macros that do not allocate new string buffers like `write!`:

```
1 use std::fmt::Write;
2 let mut buffer = String::with_capacity(50);
3 write!(&mut buffer, "s3: len = {}, number of chars = {}", s3.len(),
  s3.chars().count()).unwrap();
```

[\(playground link\)](#)

The `Display` trait uses `write!` to display printable data. (A bit like `repr` in Python.)

7.9. Vec type

This is **heap allocated** and almost the same as `String` but:

- without UTF grapheme cluster limitations
- with arbitrary data types

7.9. Vec type

This is **heap allocated** and almost the same as `String` but:

- without UTF grapheme cluster limitations
- with arbitrary data types

Example of the heapless version (not essential for this course):

```
1 use heapless::Vec; // fixed capacity `std::Vec`  
2  
3 // on the stack  
4 let mut xs: Vec<u8, 8> = Vec::new(); // can hold up to 8 elements  
5 xs.push(42)?;  
6 assert_eq!(xs.pop(), Some(42));
```

([playground link](#))

Note that we need a compile-time const upper-bound on the storage size (8).

7.10. Practice: Exercism Nucleotide Count

Exercism: Nucleotide Count:

- Path: `exercises/practice/nucleotide-count/`
- Goal: Count DNA nucleotides and handle invalid inputs
- Requires: `Result`, `HashMap`, error handling, iterators

Online: <https://exercism.org/tracks/rust/exercises/nucleotide-count>

Warning

Important Exercise

Please solve exercise `session-2/examples/s2d8-counter.rs`

1. Quiz about last lecture	1
2. Pattern matching	8
3. Tooling intermezzo	25
4. Methods and traits	29
5. Generics	48
6. Closures	69
7. Standard library types	85
8. Conclusion	111

Homework:

- study the standard library API and helper functions of `Result`
- study standard library traits
- (optional) read the `unsafe` chapter of Blandy and discussion about making safe buffer

Questions?

Feel free to contact me:

- Video meeting / email: willemvanhulle@protonmail.com
- Phone: +32 479 080 252